

Wealth Analytics Data Platform

Enterprise Azure Data Engineering — Implementation Report

Field	Detail
Project Type	Azure Data Engineering Case Study Implementation
Organization	FinWealth Advisors — Wealth Management Analytics Platform
Document Type	Technical Architecture & Implementation Documentation
Prepared For	Case Study: “Building a Wealth Analytics Data Platform”
Prepared By	Anjali Kolambkar

Implementation Areas

Azure Service / Area
Azure Data Lake Storage Gen2 (ADLS Gen2)
Azure SQL Database
Azure Data Factory (ADF)
ETL / ELT Architecture & Orchestration
Data Warehousing — Star Schema Modeling
Medallion Architecture (Bronze / Silver / Gold)
Data Validation, Logging & Monitoring
Financial Analytics Platform Design

Table of Contents

Table of Contents	2
Phase 1 — Project Overview, Business Understanding & High-Level Architecture	8
1. Project Overview	8
1.1 Project Introduction	8
1.2 Business Context	8
2. Business Problem Statement	8
2.1 Existing Challenges	8
2.2 Required Business Outcomes	9
3. Project Objectives	9
3.1 Primary Technical Objectives	9
3.2 Analytical Objectives	10
4. Dataset Inventory	10
4.1 Source Dataset Overview	10
5. Business Understanding of Datasets	10
5.1 Master Datasets	10
5.2 Transactional Datasets	11
5.3 Analytical & Market Datasets	11
6. High-Level Architecture	11
6.1 Proposed Enterprise Architecture	11
6.2 Core Azure Services	12
7. Initial Data Engineering Strategy	13
7.1 Data Profiling-First Approach	13
7.2 Dataset Analysis Strategy	13
8. Enterprise Architecture Decisions	13
8.1 ELT-Oriented Architecture	13
9. Cost Optimization Considerations	14
9.1 Cost-Safe Engineering Approach	14
10. Initial Warehouse Thinking	14
10.1 Fact & Dimension Candidates	14
11. Architecture Notes	15
Medallion Readiness	15
Data Governance	15

Separation of Concerns	15
12. Phase 1 Summary	15
Phase 2 — Azure Setup, ADLS Gen2, Medallion Architecture & Storage Design.....	16
13. Azure Resource Configuration.....	16
13.1 Resource Group Setup.....	16
13.2 Enterprise Naming Convention Standard	16
14. ADLS Gen2 Storage Account Configuration.....	16
14.1 Storage Account Settings.....	16
14.2 Rationale for ADLS Gen2 Selection.....	17
14.3 Storage Security Configuration.....	17
15. Medallion Architecture Design	17
15.1 Architecture Overview.....	17
15.2 Bronze Layer Design	19
15.3 Silver Layer Design.....	19
15.4 Gold Layer Design	20
16. Storage Architecture Decision Rationale.....	20
17. Phase 2 Summary	20
Phase 3 — Data Profiling, Data Quality, Cleaning & Warehouse Design	22
18. Data Profiling Strategy.....	22
18.1 Profiling Overview	22
18.2 Profiling Objectives.....	22
18.3 Dataset Profiling Sequence.....	22
19. Data Quality Analysis	23
19.1 Enterprise Data Quality Framework.....	23
19.2 Data Quality Issues Identified.....	23
20. Data Cleaning Strategy	24
20.1 Layered Cleansing Philosophy	24
21. ELT Architecture Decision.....	24
21.1 Why ELT Was Selected.....	24
22. Data Warehouse Design	25
22.1 Warehouse Design Philosophy	25
22.2 Why Star Schema Was Selected	25
23. Fact & Dimension Identification	25
23.1 Fact Table Candidates.....	25

23.2 Dimension Candidates	26
24. Grain Definition	26
24.1 Importance of Grain Definition	26
25. Surrogate Keys & Slowly Changing Dimensions	27
25.1 Surrogate Key Strategy	27
25.2 SCD Design Approach	27
26. Phase 3 Summary	27
Phase 4 — SQL Design, Staging Layer, Fact/Dimension Modeling, SCD & Surrogate Keys	28
27. SQL Layer Architecture	28
27.1 Purpose of the SQL Layer.....	28
27.2 SQL Schema Design.....	28
28. Staging Layer Design.....	28
28.1 Staging Layer Purpose	28
28.2 Staging Naming Convention	29
29. Master Staging Tables	29
29.1 stg.Clients	29
29.2 stg.Advisors.....	30
29.3 stg.Accounts	30
29.4 stg.Securities.....	30
29.4 stg.Benchmarks	31
30. Transactional Staging Tables	31
30.1 stg.Trades	31
30.2 stg.Holdings	31
30.3 stg.Performance	32
30.4 stg.Cashflows	32
30.5 stg.Fees	32
30.6 stg.Prices.....	33
31. Warehouse Dimension Table Design.....	33
31.1 Dimension Design Principles.....	33
31.2 dwh.DimClient	33
31.3 dwh.DimAdvisor	34
31.4 dwh.DimAccount	34
31.5 dwh.DimSecurity	34
31.6 dwh.DimBenchmark	34

32. Warehouse Fact Table Design	35
32.1 Fact Table Design Principles	35
32.2 dwh.FactTrades	35
32.3 dwh.FactHoldings	35
32.4 dwh.FactPerformance	36
32.5 dwh.FactCashflows	36
32.6 dwh.FactFees	37
33. Fact-Dimension Relationship Design	37
34. Dimension Loading Stored Procedures	38
34.1 Loading Strategy	38
34.2 dwh.SP_Load_DimClient.....	38
34.3 dwh.SP_Load_DimAdvisor.....	39
34.4 dwh.SP_Load_DimAccount.....	40
34.5 dwh.SP_Load_DimSecurity	40
35. Fact Loading Stored Procedures	41
35.1 Loading Strategy	41
35.2 dwh.SP_Load_FactTrades.....	41
35.3 dwh.SP_Load_FactHoldings.....	42
35.4 dwh.SP_Load_FactCashflows	42
35.5 dwh.SP_Load_FactFees	43
35.6 dwh.SP_Load_FactPerformance.....	43
36. SQL Validation Queries	44
36.1 Row Count Validation	44
36.2 Duplicate Checks.....	44
36.3 Null Checks	44
36.4 Referential Integrity Checks	44
36.5 Business Rule Checks	45
36.6 Source-to-Target Mapping	45
37. Load Orchestration Dependency.....	46
38. Phase 4 Summary	46
Phase 5 — Azure Data Factory Pipelines, Validation Logic, Logging & Monitoring	47
39. Azure Data Factory Architecture	47
39.1 ADF Purpose and Role	47
39.2 ADF Configuration	47

40. Linked Services	47
40.1 Linked Service Inventory.....	47
40.2 SQL Connectivity Troubleshooting	48
41. Dataset Design in ADF	48
41.1 Dataset Naming Convention.....	48
42. Bronze-to-Staging Pipeline Design	49
42.1 Pipeline Inventory.....	49
42.2 Copy Activity Configuration Standard	50
43. Pipeline Validation Steps	50
44. Transformation Pipeline Design	50
44.1 Dimension Load Pipeline	50
44.2 Fact Load Pipeline.....	51
45. Master Pipeline Orchestration	52
45.1 Master Pipeline: PL_Master_Load_DWH	52
46. Validation Logic	53
46.1 Validation Framework	53
46.2 Row Reconciliation Queries.....	53
47. Validation Stored Procedure	53
47.1 audit.ValidationResults Table	53
47.2 audit.SP_Validate_StagingData	54
48. Logging Framework	55
48.1 audit.PipelineRunLog Table	55
48.2 audit.SP_LogPipelineStart	55
48.3 audit.SP_LogPipelineEnd	55
48.4 ADF Dynamic Content References.....	56
49. Monitoring Strategy.....	56
49.1 Operational Monitoring Queries	57
50. Error Handling & Retry Strategy	57
51. Phase 5 Summary	57
Phase 6 — Business Rules, Cost Optimization & Enterprise Best Practices	58
52. Business Rules Framework	58
52.1 Business Rule Categories	58
53. Entity-Level Business Rules.....	58
53.1 Client Business Rules	58

53.2 Account Business Rules	58
53.3 Trade Business Rules	59
53.4 Holdings, Performance & Financial Business Rules	59
54. Data Quality Governance Rules	60
55. Cost Optimization Strategy	60
56. Enterprise Best Practices Summary	61
56.1 Architecture Best Practices	61
56.2 Security & Governance	61
56.3 Troubleshooting Reference	61
57. Phase 6 Summary	61
Appendix D -SQL Queries Used During Practical Implementation	63
Appendix — Project Repository & References	72
A.1 GitHub Repository	72

Phase 1 — Project Overview, Business Understanding & High-Level Architecture

1. Project Overview

1.1 Project Introduction

The Wealth Analytics Data Platform was designed as a cloud-based Azure Data Engineering solution for FinWealth Advisors, a wealth management organization. The implementation objective was to design and deploy a centralized analytics platform integrating multiple operational datasets into a scalable analytical environment for business intelligence and wealth analytics reporting.

The implementation encompassed the following areas:

- Centralized financial data ingestion
- Enterprise data warehousing
- ETL/ELT orchestration
- Cloud storage architecture
- Analytical modeling
- Scalable reporting preparation

The solution architecture was built using Azure Data Lake Storage Gen2 (ADLS Gen2), Azure SQL Database, Azure Data Factory (ADF), SQL-based transformations, and Medallion Architecture principles.

1.2 Business Context

FinWealth Advisors operates across multiple wealth management functions including portfolio advisory, investment management, transaction management, commission tracking, performance analysis, and client advisory operations. Operational data was distributed across disconnected systems including CRM platforms, transactional systems, market data feeds, advisor systems, and commission systems. The absence of centralized analytical infrastructure created challenges in consolidated reporting, performance analytics, advisor profitability analysis, AUM reporting, and client-level intelligence generation.

2. Business Problem Statement

2.1 Existing Challenges

The organization lacked a centralized enterprise data platform capable of consolidating operational financial datasets, supporting scalable analytics, enabling historical tracking, and generating business-ready analytical datasets.

Problem Area	Business Impact
Distributed source systems	Inconsistent reporting
No centralized warehouse	Limited analytics scalability
Manual reporting dependencies	Slow business insights
Data quality inconsistencies	Reduced reporting reliability
No standardized transformation layer	Duplicate business logic
Lack of historical tracking	Incomplete analytical visibility
No governed ETL architecture	Operational inefficiencies

2.2 Required Business Outcomes

The organization required a centralized platform delivering the following analytical outputs:

- Assets Under Management (AUM) analytics
- Advisor performance reporting
- Client profitability analysis
- Product mix analytics
- Commission tracking
- Portfolio performance insights
- Analytical datasets for Power BI reporting

3. Project Objectives

3.1 Primary Technical Objectives

Objective	Description
Centralized Storage	Establish Azure-based centralized storage architecture
Data Ingestion	Build scalable ingestion pipelines from source to staging
Data Warehousing	Implement star-schema warehouse design
Data Transformation	Design SQL-based transformation framework
Validation Framework	Implement enterprise validation logic
Monitoring	Establish logging & monitoring strategy
Analytics Readiness	Prepare reporting-ready datasets
Cost Optimization	Maintain cloud cost efficiency

3.2 Analytical Objectives

Target analytical outputs included AUM reporting, advisor-level performance analysis, client profitability analysis, product category reporting, benchmark comparison analytics, portfolio performance reporting, fee analysis, and cashflow analysis.

4. Dataset Inventory

4.1 Source Dataset Overview

The project utilized a medium-scale financial analytics dataset package consisting of master, transactional, market, and performance datasets.

Dataset	Approx. Rows	Business Purpose
Clients	8,000	Client master data
Accounts	12,000	Investment account data
Advisors	350	Advisor master data
Securities	3,000	Financial instrument reference
Trades	100,400	Transactional trade events
Holdings	96,027	Portfolio holdings snapshots
Prices	301,500	Daily market prices
Performance	110,000	Portfolio performance metrics
Cashflows	30,000	Deposit / withdrawal activity
Fees	18,000	Fee & commission transactions
Benchmarks	4,020	Benchmark index reference
Data Dictionary	64	Metadata & field mapping

5. Business Understanding of Datasets

5.1 Master Datasets

Clients

Represents investor/customer master data including client identifiers, segmentation, risk profiles, and demographic information.

Advisors

Contains wealth advisor reference information including advisor identifiers, regional mapping, and advisor assignment details.

Securities

Represents financial instruments including stocks, bonds, ETFs, and investment products.

5.2 Transactional Datasets**Trades**

Represents buy/sell transactional events and forms one of the primary fact-table candidates.

Cashflows

Represents deposits, withdrawals, and account-level cash movement events.

Fees

Represents advisory and transaction fee events used for profitability and commission analysis.

5.3 Analytical & Market Datasets**Holdings**

Represents account-level portfolio holdings snapshots.

Prices

Contains daily security pricing information used for valuation and performance analytics.

Performance

Contains portfolio/account performance metrics.

Benchmarks

Contains market benchmark index values used for comparison analytics.

6. High-Level Architecture**6.1 Proposed Enterprise Architecture**

The solution architecture followed a layered Azure Data Engineering design pattern, progressing from raw source ingestion through staged transformation to analytical consumption.

WEALTH ANALYTICS DATA PLATFORM – HIGH LEVEL ARCHITECTURE

Enterprise Azure Data Engineering Architecture – End to End Flow

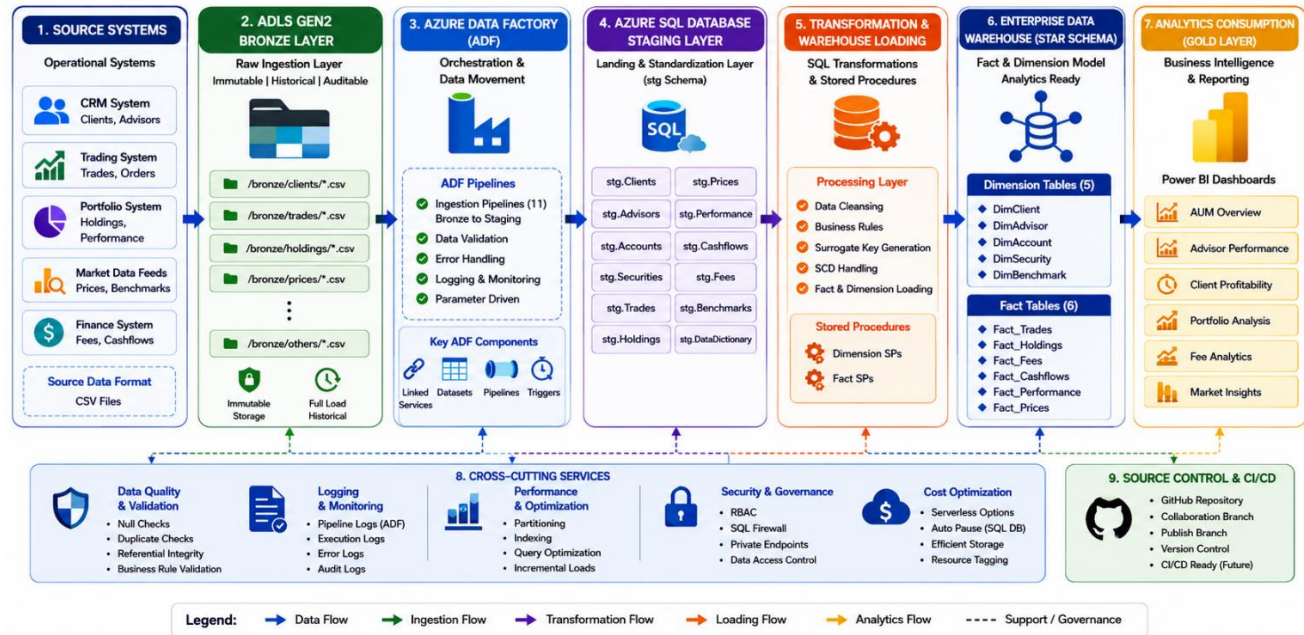


Figure 1.1 — Wealth Analytics Platform High-Level Architecture

Layer / Component	Description
Source CSV Files	Raw operational datasets from source systems
ADLS Gen2 — Bronze Layer	Centralized immutable raw storage
ADF Ingestion Pipelines	Orchestrated file-to-staging movement
Azure SQL Staging Layer	Controlled SQL landing and validation zone
SQL Transformations & Stored Procedures	Business-rule enforcement and cleansing
Dimension & Fact Tables	Star-schema warehouse layer
Gold Analytical Layer	Business-ready curated datasets
Power BI Reporting	Dashboard and analytics consumption

6.2 Core Azure Services

Azure Service	Purpose in Solution
ADLS Gen2	Centralized cloud storage (Data Lake)
Azure SQL Database	Staging and warehouse processing layer

Azure Service	Purpose in Solution
Azure Data Factory	ETL/ELT orchestration and monitoring
Power BI	Reporting and analytical visualization

7. Initial Data Engineering Strategy

7.1 Data Profiling-First Approach

The implementation intentionally prioritized source system understanding, profiling, and data quality analysis before commencing warehouse modeling. This approach aligned with real enterprise Data Engineering methodology where structural analysis and relationship discovery must precede analytical architecture design.

7.2 Dataset Analysis Strategy

Analysis Area	Purpose
Structural Profiling	Column & datatype analysis
Relationship Discovery	PK/FK candidate identification
Data Quality Analysis	Nulls, duplicates, inconsistencies
Warehouse Suitability	Fact/dimension classification
Transformation Planning	Cleansing & validation strategy

8. Enterprise Architecture Decisions

8.1 ELT-Oriented Architecture

The implementation primarily followed an ELT-based cloud architecture strategy — Extract, Load, then Transform — rather than traditional ETL. SQL-centric transformations were intentionally preferred over excessive ADF Mapping Data Flow complexity.

Reason	Explanation
Cost efficiency	SQL transformations are more cost-effective than Mapping Data Flow Debug sessions
Scalability	The SQL layer is straightforward to scale for growing datasets
Maintainability	Stored procedures support reusable and version-controlled logic

Reason	Explanation
Debugging simplicity	SQL scripts are easier to inspect and troubleshoot than visual pipelines
Enterprise alignment	Warehouse-centric processing is a standard cloud Data Engineering pattern

9. Cost Optimization Considerations

9.1 Cost-Safe Engineering Approach

Service	Decision / Reason
ADLS Gen2	Standard tier with LRS — lowest-cost scalable storage
Azure SQL Database	Basic / low-tier compute — sufficient for project scale
ADF Copy Activities	Preferred — lightweight and cost-safe
SQL Transformations	Preferred over Mapping Data Flows — no cluster compute cost
Azure Databricks	Intentionally avoided — unnecessary compute overhead
Azure Synapse	Intentionally avoided — excessive for project scope
Mapping Data Flow Debug	Intentionally avoided — hidden Integration Runtime costs

10. Initial Warehouse Thinking

10.1 Fact & Dimension Candidates

Dataset	Warehouse Role
Trades	Fact — transactional trade events
Holdings	Fact — portfolio snapshot data
Performance	Fact — measurable performance metrics
Fees	Fact — financial fee events
Cashflows	Fact — cash movement transactions
Clients	Dimension — client master reference
Accounts	Dimension — account master reference
Advisors	Dimension — advisor master reference

Dataset	Warehouse Role
Securities	Dimension — financial instrument reference
Benchmarks	Dimension — benchmark index reference

11. Architecture Notes

Medallion Readiness

The architecture was aligned toward Bronze, Silver, and Gold data lake layering principles to support future scalability and modern lakehouse alignment.

Data Governance

Raw datasets were intentionally preserved unchanged to support auditability, lineage tracking, rollback capability, and reproducibility.

Separation of Concerns

The architecture separated storage, transformation, orchestration, and analytics into independent layers for maintainability and scalability.

12. Phase 1 Summary

Phase 1 established the foundational engineering strategy covering business understanding, enterprise architecture direction, source-system analysis, high-level warehouse thinking, and initial cloud architecture planning. The implementation prioritized structured profiling, governed ingestion, scalable transformation design, and enterprise-grade warehouse planning before commencing pipeline implementation.

Phase 2 — Azure Setup, ADLS Gen2, Medallion Architecture & Storage Design

13. Azure Resource Configuration

13.1 Resource Group Setup

The Azure Resource Group served as the centralized logical container for all cloud infrastructure resources.

Configuration	Value
Resource Group Name	rg-finwealth-dev
Environment Type	Development
Azure Region	Central India
Subscription Type	Azure for Students

13.2 Enterprise Naming Convention Standard

Resource Type	Naming Pattern
Resource Group	rg-<project>-<env>
Storage Account	st<project><unique>
Data Factory	adf-<project>-<env>
SQL Database	sqldb-<project>-<env>

Note: Enterprise cloud environments typically maintain Development, Test, UAT, and Production resource segregation. A Development environment was deployed first to reduce complexity, minimize operational cost, and streamline architecture learning.

14. ADLS Gen2 Storage Account Configuration

14.1 Storage Account Settings

Configuration	Value
Storage Account Name	Stfinwealthanjali
Storage Type	General Purpose v2

Configuration	Value
Performance Tier	Standard
Redundancy	LRS (Locally Redundant Storage)
Region	Central India
Hierarchical Namespace	Enabled

14.2 Rationale for ADLS Gen2 Selection

ADLS Gen2 was selected for its scalable cloud storage, hierarchical directory management, big data readiness, enterprise lake architecture support, and analytical workload optimization. Enabling the hierarchical namespace transformed the storage account into a Data Lake-optimized architecture supporting Medallion architecture, scalable analytical processing, and enterprise governance patterns.

14.3 Storage Security Configuration

Setting	Value
Anonymous Access	Private — disabled
Public Blob Access	Disabled
Encryption Scope	Default (platform-managed)
Immutable Storage	Disabled initially

Note: Private access was enforced because financial datasets contain sensitive business data. Public access introduces unacceptable security risk in wealth management environments.

15. Medallion Architecture Design

15.1 Architecture Overview

The storage architecture followed the Medallion Architecture pattern widely used across Azure Data Engineering, Databricks, Snowflake, Microsoft Fabric, and enterprise lakehouse implementations.

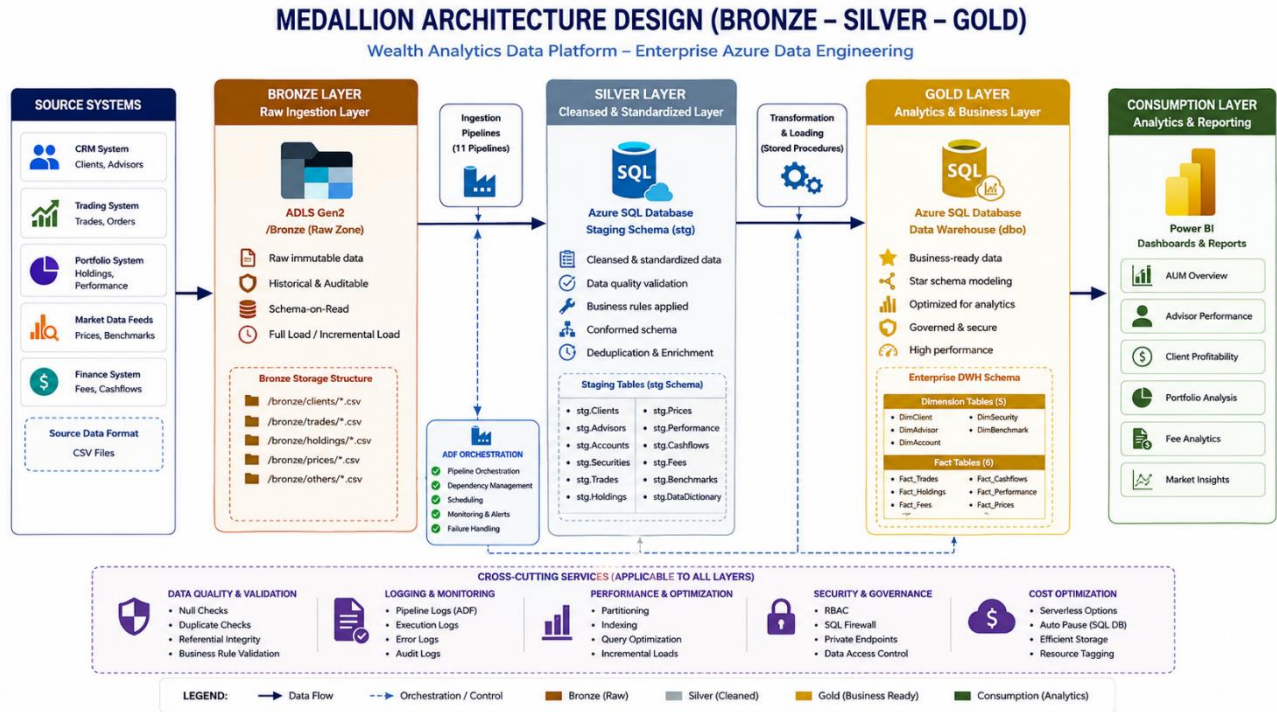


Figure 2.1 — Medallion Architecture Data Flow

Layer	Purpose	Characteristics
Bronze	Raw immutable source data	Immutable, auditable, reprocessable, historical
Silver	Cleaned and validated datasets	Standardized, validated, transformation-ready
Gold	Business-ready analytical datasets	Curated, optimized, reporting-consumable
Archive	Historical retention & rollback	Processed-file storage for replay support
Logs	ETL execution logs & monitoring	Validation reports, row counts, rejected records

15.2 Bronze Layer Design

The Bronze layer was designed to preserve raw source files as the system-of-truth, supporting auditability and reprocessing capability. The following datasets were loaded into the Bronze container:

Dataset	File Format
Accounts	CSV
Advisors	CSV
Benchmarks	CSV
Cashflows	CSV
Clients	CSV
Data Dictionary	CSV
Fees	CSV
Holdings	CSV
Performance	CSV
Prices	CSV
Securities	CSV
Trades	CSV

Note: Raw source datasets were not renamed, cleaned, deduplicated, or transformed inside the Bronze layer. This governance rule aligned with enterprise data standards for lineage preservation and auditability.

15.3 Silver Layer Design

The Silver layer was designed as the cleansing, validation, transformation, and standardization zone. It contained validated datasets, standardized formats, and transformation-ready analytical entities.

Transformation Type	Description
Null handling	Standardization and controlled replacement
Datatype correction	Consistent schema enforcement
Duplicate handling	Deduplication logic
Business rule validation	Financial rule enforcement

Transformation Type	Description
Data standardization	Controlled formatting and categorization

15.4 Gold Layer Design

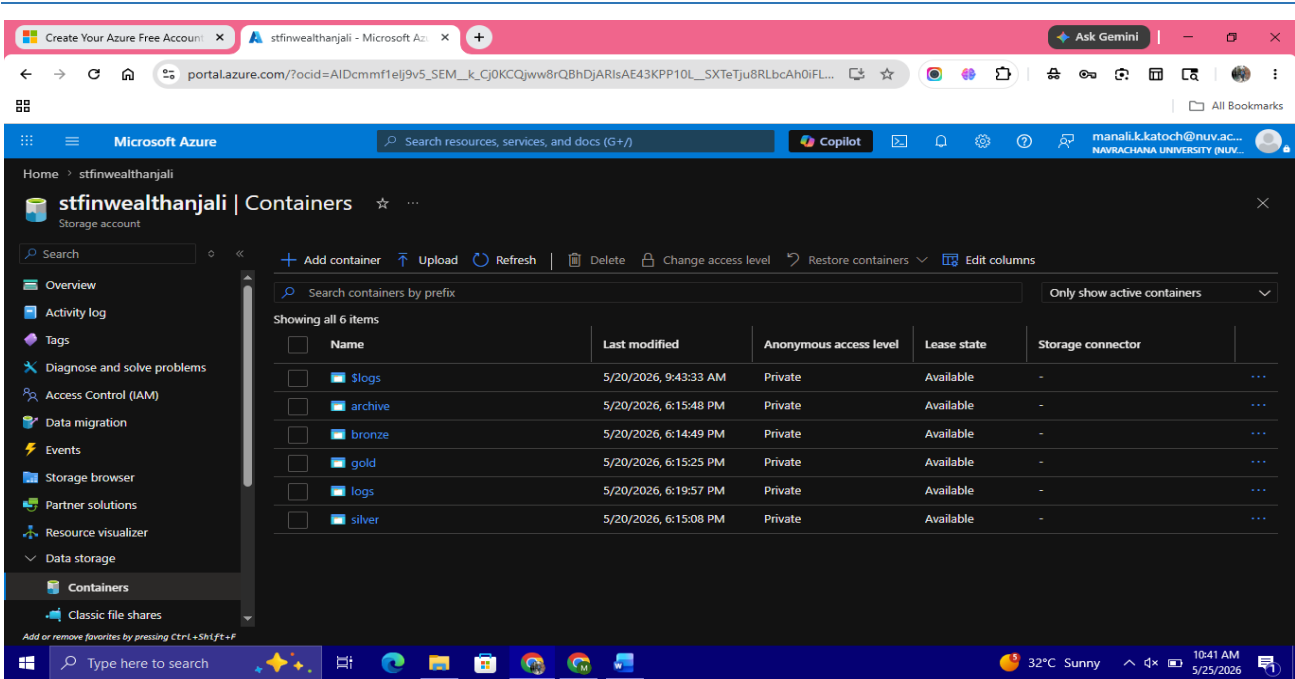
The Gold layer was designed for business-ready analytics, reporting consumption, KPI calculation, and Power BI integration. Gold outputs included AUM Summary, Advisor Performance, Client Profitability, Product Mix Analytics, and Fee Analytics datasets.

16. Storage Architecture Decision Rationale

Benefit	Explanation
Reprocessing	Raw files preserved in Bronze allow full ETL replay
Scalability	Decoupled storage and compute supports independent scaling
Auditability	Source history is maintained for regulatory and debugging needs
Governance	Centralized ingestion through a single governed storage layer
Maintainability	Clear layer boundaries simplify troubleshooting

17. Phase 2 Summary

Phase 2 established the Azure infrastructure setup, ADLS Gen2 configuration, Medallion architecture implementation, enterprise storage strategy, governance foundations, and scalable layered storage design. This phase created the foundational cloud architecture required for staging layer implementation, SQL warehouse design, ADF pipeline orchestration, and downstream analytical processing.



Phase 3 — Data Profiling, Data Quality, Cleaning & Warehouse Design

18. Data Profiling Strategy

18.1 Profiling Overview

The implementation prioritized enterprise-style data profiling before warehouse modeling, ETL pipeline development, or transformation logic implementation. This approach aligned with real-world Data Engineering methodology where source-system understanding and data quality assessment must precede analytical architecture design.

18.2 Profiling Objectives

Profiling Area	Objective
Structural Analysis	Understand schema structure and column characteristics
Relationship Discovery	Identify PK/FK candidates and entity relationships
Data Quality Assessment	Detect nulls, duplicates, and invalid values
Warehouse Suitability	Determine fact vs. dimension classification
Transformation Planning	Identify cleansing and validation requirements
Business Understanding	Understand analytical meaning and usage context
Validation Planning	Define rule enforcement logic for warehouse loading

18.3 Dataset Profiling Sequence

Datasets were analyzed in dependency order to simplify relationship discovery and dimensional modeling:

Sequence	Dataset	Rationale
1	Data Dictionary	Metadata baseline
2	Clients	Core master entity
3	Advisors	Master reference
4	Accounts	Depends on Clients
5	Securities	Financial instrument reference
6	Benchmarks	Analytical comparison reference

Sequence	Dataset	Rationale
7	Trades	Depends on Accounts & Securities
8	Holdings	Depends on Accounts & Securities
9	Prices	Depends on Securities
10	Performance	Depends on Accounts & Benchmarks
11	Cashflows	Depends on Accounts
12	Fees	Depends on Accounts

19. Data Quality Analysis

19.1 Enterprise Data Quality Framework

The project evaluated data quality across structural, business, and analytical dimensions to ensure the warehouse architecture was based on governed datasets with validated business relationships.

Validation Area	Purpose
Datatype Validation	Ensure schema consistency
Null Analysis	Detect missing mandatory data
Duplicate Detection	Validate entity uniqueness
Date Validation	Ensure temporal consistency
Numeric Validation	Detect invalid amounts and metrics
Categorical Validation	Standardize business values
Referential Integrity	Validate entity relationships

19.2 Data Quality Issues Identified

Issue Type	Example	Planned Resolution
Missing Values	Null client attributes	Standardization or flagging
Duplicate Records	Duplicate transaction IDs	Deduplication logic
Invalid Formats	Incorrect date formats	Conversion and rejection logic
Relationship Gaps	Missing foreign keys	Referential validation

Issue Type	Example	Planned Resolution
Inconsistent Categories	Gender/region inconsistencies	Controlled value mappings
Numeric Issues	Negative financial amounts	Business rule validation
Standardization	Mixed formatting conventions	Transformation standardization

20. Data Cleaning Strategy

20.1 Layered Cleansing Philosophy

The implementation followed a layered cleansing strategy. Raw datasets remained immutable inside the Bronze layer, with cleansing responsibilities distributed across appropriate architecture layers.

Transformation Type	Responsible Layer
File Ingestion	ADF Copy Activity
Basic Profiling	Power Query / Initial Analysis
Datatype Standardization	SQL Staging Layer
Deduplication	SQL Transformation Layer
Business Rule Validation	SQL Stored Procedures
Referential Integrity	SQL Validation Procedures
Analytical Aggregation	Gold Layer

Note: Direct modification of Bronze datasets would break data lineage, reduce debugging capability, and violate enterprise governance principles. The Bronze layer intentionally preserves raw source history for auditability and reproducibility.

21. ELT Architecture Decision

21.1 Why ELT Was Selected

The implementation primarily followed an ELT-based architecture — Extract, Load, Transform — rather than traditional ETL.

Benefit	Explanation
Lower Azure cost	SQL transformations are more cost-effective than compute-heavy Data Flows
Easier debugging	Centralized, query-based troubleshooting
Reusability	Stored procedures can be reused across multiple pipeline runs
Scalability	Warehouse-driven processing scales with the data tier
Governance	Business logic is centralized within the SQL transformation layer

22. Data Warehouse Design

22.1 Warehouse Design Philosophy

The warehouse architecture followed a star-schema, analytical, reporting-optimized design. The design prioritized reporting simplicity, Power BI usability, scalability, and governed business logic.

22.2 Why Star Schema Was Selected

Benefit	Explanation
Simpler reporting	Straightforward join patterns for analytical queries
Faster analytics	Optimized aggregation paths
Better Power BI integration	Simplified semantic model design
Business readability	Clear separation of descriptive and measurable entities
Scalability	Modular design supports incremental expansion

23. Fact & Dimension Identification

23.1 Fact Table Candidates

Dataset	Warehouse Role	Rationale
Trades	Fact	Transactional trade events — measurable events
Holdings	Fact	Portfolio holding snapshots — periodic observations
Performance	Fact	Measurable account performance metrics
Fees	Fact	Financial fee events
Cashflows	Fact	Cash movement transactions

23.2 Dimension Candidates

Dataset	Warehouse Role	Rationale
Clients	Dimension	Business entity — descriptive client attributes
Advisors	Dimension	Descriptive advisor reference
Accounts	Dimension	Investment account descriptive attributes
Securities	Dimension	Financial instrument reference data
Benchmarks	Dimension	Analytical comparison entity

24. Grain Definition

24.1 Importance of Grain Definition

Grain definition was one of the most critical warehouse design activities. Incorrect grain leads to incorrect analytics, duplication, aggregation problems, and reporting inconsistency. The implementation intentionally defined business grain before warehouse structure design.

Dataset	Defined Grain
Clients	One row per unique client
Advisors	One row per unique advisor
Accounts	One row per unique account
Trades	One row per trade transaction
Holdings	One row per account-security-date combination
Prices	One row per security-date combination
Performance	One row per account-date performance observation
Fees	One row per fee event
Cashflows	One row per cash movement event

25. Surrogate Keys & Slowly Changing Dimensions

25.1 Surrogate Key Strategy

Each dimension table used an integer surrogate key generated by the warehouse. Business keys were retained from source systems for traceability.

Reason	Explanation
Join optimization	Integer keys outperform text-based joins
Source independence	Warehouse logic is decoupled from source-system key changes
SCD support	Enables historical tracking without business-key conflicts
Standardization	Consistent warehouse relationship design

25.2 SCD Design Approach

Several business entities were identified as requiring historical tracking for financial analytics — notably client advisor assignment, risk profile, and region changes. The DimClient table included SCD Type 2 support columns.

SCD Type	Planned Usage
Type 1 — Overwrite	Non-historical corrections (minor attribute fixes)
Type 2 — Historical rows	Advisor assignment, risk profile, region changes

Note: In wealth management, historical changes can affect analytical interpretation. Without SCD handling, historical reporting may incorrectly reflect current values instead of values valid at the time of the observed event.

26. Phase 3 Summary

Phase 3 established the enterprise data profiling methodology, warehouse analytical reasoning, data quality strategy, cleansing architecture, dimensional modeling direction, and validation framework planning. This phase formed the foundational analytical reasoning layer required before SQL implementation, ETL orchestration, and Azure Data Factory pipeline development.

Phase 4 — SQL Design, Staging Layer, Fact/Dimension Modeling, SCD & Surrogate Keys

27. SQL Layer Architecture

27.1 Purpose of the SQL Layer

The SQL layer served as the core processing and warehouse foundation of the Wealth Analytics Data Platform. Azure SQL Database supported staging table storage, data validation, transformation logic, dimension loading, fact loading, surrogate key management, business rule enforcement, and analytical warehouse preparation.

27.2 SQL Schema Design

The SQL database design separated objects into logical schemas to improve maintainability and enterprise clarity.

Schema	Purpose
stg	Raw/staging tables loaded from Bronze CSV files
Dwh	Final dimension and fact warehouse tables
audit	Logging, validation, and ETL monitoring tables

Schema Creation Script

```
CREATE SCHEMA stg;  
GO  
  
CREATE SCHEMA dwh;  
GO  
  
CREATE SCHEMA audit;  
GO
```

28. Staging Layer Design

28.1 Staging Layer Purpose

The staging layer acted as the controlled landing zone inside Azure SQL Database. Its purpose was to receive data from the ADLS Gen2 Bronze layer via Azure Data Factory and prepare it for validation and transformation.

Responsibility	Description
Raw SQL landing	Store Bronze CSV files in relational SQL format
Datatype enforcement	Convert source text into controlled SQL datatypes
Load tracking	Capture LoadDate metadata for audit purposes
Validation support	Enable null, duplicate, and referential integrity checks
Transformation preparation	Prepare data for dimension and fact loading
Debugging support	Allow direct SQL inspection post-ingestion

28.2 Staging Naming Convention

Staging tables followed the pattern `stg.<DatasetName>` — for example, `stg.Clients`, `stg.Trades`, `stg.Holdings`.

29. Master Staging Tables

29.1 `stg.Clients`

Table Definition

```
CREATE TABLE stg.Clients
(
    ClientID VARCHAR(50),
    ClientName NVARCHAR(200),
    Segment VARCHAR(100),
    RiskProfile VARCHAR(100),
    PrimaryAdvisorID VARCHAR(50),
    ClientSince DATE,
    Country VARCHAR(100),
    LoadDate DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

29.2 stg.Advisors

```
CREATE TABLE stg.Advisors
(
    AdvisorID VARCHAR(50),
    AdvisorName NVARCHAR(200),
    Region VARCHAR(100),
    JoinDate DATE,
    ExperienceYears INT,
    LoadDate DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

29.3 stg.Accounts

```
CREATE TABLE stg.Accounts
(
    AccountID VARCHAR(50),
    ClientID VARCHAR(50),
    AccountType VARCHAR(100),
    OpenDate DATE,
    BaseCurrency VARCHAR(10),
    Status VARCHAR(50),
    LoadDate DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

29.4 stg.Securities

```
CREATE TABLE stg.Securities
(
    SecurityID VARCHAR(50),
    Ticker VARCHAR(50),
    SecurityName NVARCHAR(200),
    AssetClass VARCHAR(100),
    Sector VARCHAR(100),
    Currency VARCHAR(10),
    Region VARCHAR(100),
    LoadDate DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

29.4 stg.Benchmarks

```
CREATE TABLE stg.Benchmarks
(
    BenchmarkCode VARCHAR(50),
    Description NVARCHAR(200),
    Currency VARCHAR(10),
    IndexValueDate DATE,
    IndexValue DECIMAL(18,4),
    LoadDate DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

Note: The LoadDate column was included in all staging tables to support ingestion tracking and auditability. During ADF column mapping, LoadDate must not be mapped from the source CSV. The SQL DEFAULT value populates it automatically.

30. Transactional Staging Tables

30.1 stg.Trades

```
CREATE TABLE stg.Trades
(
    TradeID          VARCHAR(50),
    AccountID        VARCHAR(50),
    SecurityID        VARCHAR(50),
    TradeDate         DATE,
    TradeType         VARCHAR(20),
    Quantity          DECIMAL(18,4),
    Price             DECIMAL(18,4),
    TradeAmount       DECIMAL(18,4),
    Broker            VARCHAR(100),
    LoadDate         DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

30.2 stg.Holdings

```
CREATE TABLE stg.Holdings
(
    HoldingID         VARCHAR(50),
    AccountID         VARCHAR(50),
    SecurityID        VARCHAR(50),
    HoldingDate        DATE,
    Quantity          DECIMAL(18,4),
    MarketValue       DECIMAL(18,4),
    LoadDate         DATETIME2 DEFAULT SYSDATETIME()
);
```

```
GO
```

30.3 stg.Performance

```
CREATE TABLE stg.Performance
(
    PerformanceID    VARCHAR(50),
    AccountID        VARCHAR(50),
    BenchmarkCode     VARCHAR(50),
    PerformanceDate   DATE,
    ReturnPct         DECIMAL(18,4),
    BenchmarkReturn   DECIMAL(18,4),
    LoadDate         DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

30.4 stg.Cashflows

```
CREATE TABLE stg.Cashflows
(
    CashflowID       VARCHAR(50),
    AccountID        VARCHAR(50),
    FlowDate          DATE,
    FlowType          VARCHAR(50),
    Amount            DECIMAL(18,4),
    Currency          VARCHAR(10),
    LoadDate         DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

30.5 stg.Fees

```
CREATE TABLE stg.Fees
(
    FeeID            VARCHAR(50),
    AccountID        VARCHAR(50),
    FeeDate          DATE,
    FeeType          VARCHAR(50),
    FeeAmount        DECIMAL(18,4),
    Currency          VARCHAR(10),
    LoadDate        DATETIME2 DEFAULT SYSDATETIME()
);
GO
```


30.6 stg.Prices

```
CREATE TABLE stg.Prices
(
    PriceID          VARCHAR(50),
    SecurityID       VARCHAR(50),
    PriceDate        DATE,
    ClosePrice       DECIMAL(18,4),
    Currency         VARCHAR(10),
    LoadDate        DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

31. Warehouse Dimension Table Design

31.1 Dimension Design Principles

Dimension tables store descriptive business attributes used for filtering, grouping, slicing, drill-down analysis, and business context enrichment. Each dimension includes an integer surrogate key generated by the warehouse.

31.2 dwh.DimClient

Attribute	Detail
Purpose	Client-level descriptive information for AUM, profitability, and risk profile analytics
Grain	One row per unique client (current record)
SCD Strategy	Type 2 — EffectiveDate / ExpiryDate / IsCurrent columns included

```
CREATE TABLE dwh.DimClient
(
    ClientKey          INT IDENTITY(1,1) PRIMARY KEY,
    ClientID           VARCHAR(50) NOT NULL,
    ClientName         VARCHAR(150),
    Gender             VARCHAR(20),
    DateOfBirth        DATE,
    Region             VARCHAR(100),
    RiskProfile        VARCHAR(50),
    AdvisorID          VARCHAR(50),
    EffectiveDate       DATE          DEFAULT CAST(GETDATE() AS DATE),
    ExpiryDate         DATE          NULL,
    IsCurrent          BIT           DEFAULT 1,
    CreatedDate        DATETIME2     DEFAULT SYSDATETIME()
);
GO
```

31.3 dwh.DimAdvisor

```
CREATE TABLE dwh.DimAdvisor
(
    AdvisorKey      INT IDENTITY(1,1) PRIMARY KEY,
    AdvisorID       VARCHAR(50) NOT NULL,
    AdvisorName     VARCHAR(150),
    Region          VARCHAR(100),
    ExperienceYears INT,
    CreatedDate     DATETIME2      DEFAULT SYSDATETIME()
);
GO
```

31.4 dwh.DimAccount

```
CREATE TABLE dwh.DimAccount
(
    AccountKey      INT IDENTITY(1,1) PRIMARY KEY,
    AccountID       VARCHAR(50) NOT NULL,
    ClientID        VARCHAR(50),
    AccountType     VARCHAR(100),
    OpenDate        DATE,
    Status          VARCHAR(50),
    Currency        VARCHAR(10),
    CreatedDate     DATETIME2      DEFAULT SYSDATETIME()
);
GO
```

31.5 dwh.DimSecurity

```
CREATE TABLE dwh.DimSecurity
(
    SecurityKey      INT IDENTITY(1,1) PRIMARY KEY,
    SecurityID       VARCHAR(50) NOT NULL,
    SecurityName     VARCHAR(200),
    SecurityType     VARCHAR(100),
    AssetClass       VARCHAR(100),
    Currency         VARCHAR(10),
    CreatedDate      DATETIME2      DEFAULT SYSDATETIME()
);
GO
```

31.6 dwh.DimBenchmark

```
CREATE TABLE dwh.DimBenchmark
(
```

```

BenchmarkKey    INT IDENTITY(1,1) PRIMARY KEY,
BenchmarkCode   VARCHAR(50) NOT NULL,
BenchmarkName   VARCHAR(200),
AssetClass      VARCHAR(100),
Currency        VARCHAR(10),
CreatedDate     DATETIME2 DEFAULT SYSDATETIME()
);
GO

```

32. Warehouse Fact Table Design

32.1 Fact Table Design Principles

Fact tables store measurable business events and reference dimensions through surrogate keys. All fact tables include warehouse-generated primary keys for internal referencing.

32.2 dwh.FactTrades

Attribute	Detail
Grain	One row per trade transaction
Dimensions Referenced	DimAccount, DimSecurity

```

CREATE TABLE dwh.FactTrades
(
    TradeKey    BIGINT IDENTITY(1,1) PRIMARY KEY,
    TradeID     VARCHAR(50),
    AccountKey  INT,
    SecurityKey INT,
    TradeDate   DATE,
    TradeType   VARCHAR(20),
    Quantity    DECIMAL(18,4),
    Price       DECIMAL(18,4),
    TradeAmount DECIMAL(18,4),
    Broker      VARCHAR(100),
    CreatedDate DATETIME2 DEFAULT SYSDATETIME()
);
GO

```

32.3 dwh.FactHoldings

Attribute	Detail
Grain	One row per account-security-date snapshot
Dimensions Referenced	DimAccount, DimSecurity

```
CREATE TABLE dwh.FactHoldings
(
    HoldingKey      BIGINT IDENTITY(1,1) PRIMARY KEY,
    HoldingID       VARCHAR(50),
    AccountKey      INT,
    SecurityKey     INT,
    HoldingDate     DATE,
    Quantity        DECIMAL(18,4),
    MarketValue     DECIMAL(18,4),
    CreatedDate     DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

32.4 dwh.FactPerformance

Attribute	Detail
Grain	One row per account-date performance observation
Dimensions Referenced	DimAccount, DimBenchmark

```
CREATE TABLE dwh.FactPerformance
(
    PerformanceKey  BIGINT IDENTITY(1,1) PRIMARY KEY,
    PerformanceID   VARCHAR(50),
    AccountKey      INT,
    BenchmarkKey    INT NULL,
    PerformanceDate DATE,
    ReturnPct       DECIMAL(18,4),
    BenchmarkReturn DECIMAL(18,4),
    CreatedDate     DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

32.5 dwh.FactCashflows

Attribute	Detail
Grain	One row per account cash movement
Dimensions Referenced	DimAccount

```
CREATE TABLE dwh.FactCashflows
(
    CashflowKey     BIGINT IDENTITY(1,1) PRIMARY KEY,
    CashflowID      VARCHAR(50),
```

```

AccountKey    INT,
FlowDate      DATE,
FlowType      VARCHAR(50),
Amount        DECIMAL(18,4),
Currency      VARCHAR(10),
CreatedDate   DATETIME2  DEFAULT SYSDATETIME()
);
GO

```

32.6 dwh.FactFees

Attribute	Detail
Grain	One row per account fee event
Dimensions Referenced	DimAccount

```

CREATE TABLE dwh.FactFees
(
    FeeKey        BIGINT IDENTITY(1,1) PRIMARY KEY,
    FeeID         VARCHAR(50),
    AccountKey    INT,
    FeeDate       DATE,
    FeeType       VARCHAR(50),
    FeeAmount     DECIMAL(18,4),
    Currency      VARCHAR(10),
    CreatedDate   DATETIME2  DEFAULT SYSDATETIME()
);
GO

```

33. Fact-Dimension Relationship Design

Fact Table	Dimensions Referenced
FactTrades	DimAccount, DimSecurity
FactHoldings	DimAccount, DimSecurity
FactPerformance	DimAccount, DimBenchmark
FactCashflows	DimAccount
FactFees	DimAccount

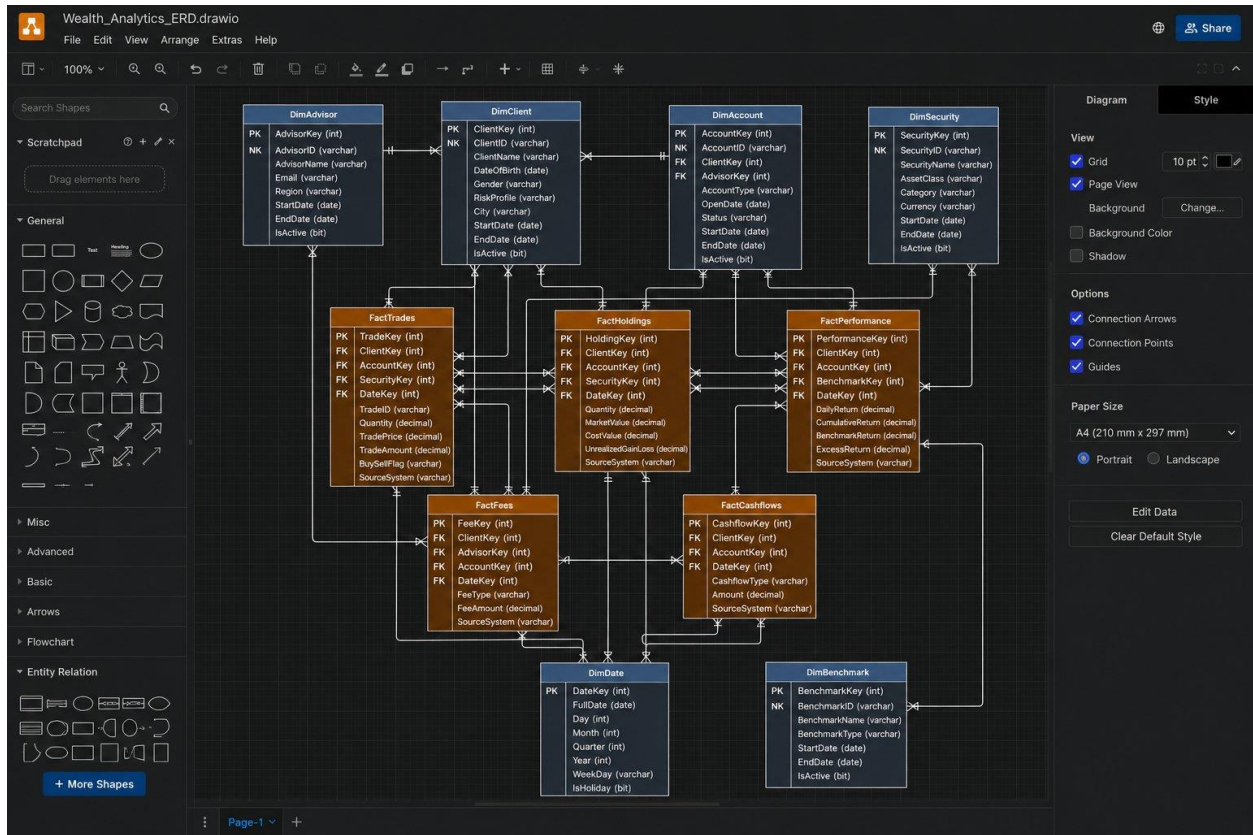


Figure 4.1 — Enterprise Star Schema for Wealth Analytics Data Platform

34. Dimension Loading Stored Procedures

34.1 Loading Strategy

Stored procedures loaded curated dimension data from staging tables into warehouse dimension tables. This strategy supported reusable transformation logic, controlled loading, pipeline integration, and simplified debugging.

34.2 dwh.SP_Load_DimClient

Attribute	Detail
Purpose	Load client dimension from stg.Clients
Source Table	stg.Clients
Target Table	dwh.DimClient
Validation Logic	WHERE NOT EXISTS — prevents duplicate IsCurrent = 1 entries
Business Rules	LTRIM/RTRIM applied to string fields; deduplication via business key + IsCurrent check

```

CREATE OR ALTER PROCEDURE dwh.SP_Load_DimClient
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dwh.DimClient
    (
        ClientID, ClientName, Gender, DateOfBirth,
        Region, RiskProfile, AdvisorID
    )
    SELECT
        LTRIM(RTRIM(ClientID)),
        LTRIM(RTRIM(ClientName)),
        LTRIM(RTRIM(Gender)),
        DateOfBirth,
        LTRIM(RTRIM(Region)),
        LTRIM(RTRIM(RiskProfile)),
        LTRIM(RTRIM(AdvisorID))
    FROM stg.Clients s
    WHERE NOT EXISTS
    (
        SELECT 1 FROM dwh.DimClient d
        WHERE d.ClientID = s.ClientID
            AND d.IsCurrent = 1
    );
END;
GO

```

34.3 dwh.SP_Load_DimAdvisor

Attribute	Detail
Purpose	Load advisor dimension from stg.Advisors
Source Table	stg.Advisors
Target Table	dwh.DimAdvisor
Validation Logic	WHERE NOT EXISTS on AdvisorID — prevents duplicate entries
Business Rules	LTRIM/RTRIM applied to string fields

```

CREATE OR ALTER PROCEDURE dwh.SP_Load_DimAdvisor
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dwh.DimAdvisor

```

```
(AdvisorID, AdvisorName, Region, ExperienceYears)
SELECT
    LTRIM(RTRIM(AdvisorID)),
    LTRIM(RTRIM(AdvisorName)),
    LTRIM(RTRIM(Region)),
    ExperienceYears
FROM stg.Advisors s
WHERE NOT EXISTS
(
    SELECT 1 FROM dwh.DimAdvisor d
    WHERE d.AdvisorID = s.AdvisorID
);
END;
GO
```

34.4 dwh.SP_Load_DimAccount

```
CREATE OR ALTER PROCEDURE dwh.SP_Load_DimAccount
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dwh.DimAccount
    (AccountID, ClientID, AccountType, OpenDate, Status, Currency)
    SELECT
        LTRIM(RTRIM(AccountID)),
        LTRIM(RTRIM(ClientID)),
        LTRIM(RTRIM(AccountType)),
        OpenDate,
        LTRIM(RTRIM(Status)),
        LTRIM(RTRIM(Currency))
    FROM stg.Accounts s
    WHERE NOT EXISTS
    (
        SELECT 1 FROM dwh.DimAccount d
        WHERE d.AccountID = s.AccountID
    );
END;
GO
```

34.5 dwh.SP_Load_DimSecurity

```
CREATE OR ALTER PROCEDURE dwh.SP_Load_DimSecurity
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dwh.DimSecurity
    (SecurityID, SecurityName, SecurityType, AssetClass, Currency)
```



```
SELECT
    LTRIM(RTRIM(SecurityID)),
    LTRIM(RTRIM(SecurityName)),
    LTRIM(RTRIM(SecurityType)),
    LTRIM(RTRIM(AssetClass)),
    LTRIM(RTRIM(Currency))
FROM stg.Securities s
WHERE NOT EXISTS
(
    SELECT 1 FROM dwh.DimSecurity d
    WHERE d.SecurityID = s.SecurityID
);
END;
GO
```

35. Fact Loading Stored Procedures

35.1 Loading Strategy

Fact tables were loaded after dimensions. This sequence was critical because fact tables depend on dimension surrogate keys. The project orchestration followed a dimension-first, fact-second loading flow, which aligns with enterprise orchestration standards.

35.2 dwh.SP_Load_FactTrades

Attribute	Detail
Purpose	Load trade fact records with resolved surrogate keys from dimensions
Source Table	stg.Trades
Target Table	dwh.FactTrades
Validation Logic	LEFT JOIN to DimAccount and DimSecurity; surrogate key resolved via business key
Business Rules	Records with unresolved surrogate keys receive NULL — should be investigated via validation procedures

```
CREATE OR ALTER PROCEDURE dwh.SP_Load_FactTrades
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dwh.FactTrades
    (TradeID, AccountKey, SecurityKey, TradeDate, TradeType,
    Quantity, Price, TradeAmount, Broker)
    SELECT
        s.TradeID,
```

```
        a.AccountKey,  
        sec.SecurityKey,  
        s.TradeDate,  
        s.TradeType,  
        s.Quantity,  
        s.Price,  
        s.TradeAmount,  
        s.Broker  
FROM stg.Trades s  
LEFT JOIN dwh.DimAccount a ON s.AccountID = a.AccountID  
LEFT JOIN dwh.DimSecurity sec ON s.SecurityID = sec.SecurityID;  
END;  
GO
```

35.3 dwh.SP_Load_FactHoldings

```
CREATE OR ALTER PROCEDURE dwh.SP_Load_FactHoldings  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    INSERT INTO dwh.FactHoldings  
    (HoldingID, AccountKey, SecurityKey, HoldingDate, Quantity, MarketValue)  
    SELECT  
        s.HoldingID,  
        a.AccountKey,  
        sec.SecurityKey,  
        s.HoldingDate,  
        s.Quantity,  
        s.MarketValue  
    FROM stg.Holdings s  
    LEFT JOIN dwh.DimAccount a ON s.AccountID = a.AccountID  
    LEFT JOIN dwh.DimSecurity sec ON s.SecurityID = sec.SecurityID;  
END;  
GO
```

35.4 dwh.SP_Load_FactCashflows

```
CREATE OR ALTER PROCEDURE dwh.SP_Load_FactCashflows  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    INSERT INTO dwh.FactCashflows  
    (CashflowID, AccountKey, FlowDate, FlowType, Amount, Currency)  
    SELECT  
        s.CashflowID,  
        a.AccountKey,  
        s.FlowDate,
```

```
s.FlowType,  
s.Amount,  
s.Currency  
FROM stg.Cashflows s  
LEFT JOIN dwh.DimAccount a ON s.AccountID = a.AccountID;  
END;  
GO
```

35.5 dwh.SP_Load_FactFees

```
CREATE OR ALTER PROCEDURE dwh.SP_Load_FactFees  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    INSERT INTO dwh.FactFees  
    (FeeID, AccountKey, FeeDate, FeeType, FeeAmount, Currency)  
    SELECT  
        s.FeeID,  
        a.AccountKey,  
        s.FeeDate,  
        s.FeeType,  
        s.FeeAmount,  
        s.Currency  
    FROM stg.Fees s  
    LEFT JOIN dwh.DimAccount a ON s.AccountID = a.AccountID;  
END;  
GO
```

35.6 dwh.SP_Load_FactPerformance

```
CREATE OR ALTER PROCEDURE dwh.SP_Load_FactPerformance  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    INSERT INTO dwh.FactPerformance  
    (PerformanceID, AccountKey, BenchmarkKey, PerformanceDate, ReturnPct,  
    BenchmarkReturn)  
    SELECT  
        s.PerformanceID,  
        a.AccountKey,  
        b.BenchmarkKey,  
        s.PerformanceDate,  
        s.ReturnPct,  
        s.BenchmarkReturn  
    FROM stg.Performance s  
    LEFT JOIN dwh.DimAccount a ON s.AccountID = a.AccountID  
    LEFT JOIN dwh.DimBenchmark b ON s.BenchmarkCode = b.BenchmarkCode;
```

```
END;  
GO
```

36. SQL Validation Queries

36.1 Row Count Validation

```
SELECT COUNT(*) AS ClientRows      FROM stg.Clients;  
SELECT COUNT(*) AS AdvisorRows     FROM stg.Advisors;  
SELECT COUNT(*) AS AccountRows     FROM stg.Accounts;  
SELECT COUNT(*) AS SecurityRows    FROM stg.Securities;  
SELECT COUNT(*) AS TradeRows       FROM stg.Trades;  
SELECT COUNT(*) AS HoldingRows     FROM stg.Holdings;  
SELECT COUNT(*) AS PerformanceRows FROM stg.Performance;  
SELECT COUNT(*) AS CashflowRows    FROM stg.Cashflows;  
SELECT COUNT(*) AS FeeRows         FROM stg.Fees;  
SELECT COUNT(*) AS PriceRows       FROM stg.Prices;
```

36.2 Duplicate Checks

```
SELECT ClientID, COUNT(*) AS DuplicateCount  
FROM   stg.Clients  
GROUP BY ClientID  
HAVING COUNT(*) > 1;  
  
SELECT TradeID, COUNT(*) AS DuplicateCount  
FROM   stg.Trades  
GROUP BY TradeID  
HAVING COUNT(*) > 1;
```

36.3 Null Checks

```
SELECT * FROM stg.Clients WHERE ClientID IS NULL;  
SELECT * FROM stg.Accounts WHERE AccountID IS NULL OR ClientID IS NULL;  
SELECT * FROM stg.Trades  
WHERE TradeID IS NULL OR AccountID IS NULL OR SecurityID IS NULL;
```

36.4 Referential Integrity Checks

```
-- Accounts with missing Client reference  
SELECT a.*  
FROM   stg.Accounts a  
LEFT   JOIN stg.Clients c ON a.ClientID = c.ClientID  
WHERE  c.ClientID IS NULL;  
  
-- Trades with missing Account reference
```

```

SELECT t.*
FROM   stg.Trades t
LEFT   JOIN stg.Accounts a ON t.AccountID = a.AccountID
WHERE  a.AccountID IS NULL;

-- Trades with missing Security reference
SELECT t.*
FROM   stg.Trades t
LEFT   JOIN stg.Security s ON t.SecurityID = s.SecurityID
WHERE  s.SecurityID IS NULL;

```

36.5 Business Rule Checks

```

SELECT * FROM stg.Trades WHERE Quantity <= 0 OR Price <= 0 OR
TradeAmount <= 0;
SELECT * FROM stg.Fees WHERE FeeAmount < 0;
SELECT * FROM stg.Prices WHERE ClosePrice <= 0;
SELECT * FROM stg.Holdings WHERE MarketValue < 0;

```

36.6 Source-to-Target Mapping

Source Table	Source Column	Target Table	Target Column
stg.Trades	TradeID	dwh.FactTrades	TradeID
stg.Trades	AccountID	dwh.FactTrades	AccountKey (via DimAccount)
stg.Trades	SecurityID	dwh.FactTrades	SecurityKey (via DimSecurity)
stg.Trades	TradeDate	dwh.FactTrades	TradeDate
stg.Trades	Quantity	dwh.FactTrades	Quantity
stg.Trades	TradeAmount	dwh.FactTrades	TradeAmount
stg.Holdings	HoldingID	dwh.FactHoldings	HoldingID
stg.Holdings	AccountID	dwh.FactHoldings	AccountKey (via DimAccount)
stg.Holdings	SecurityID	dwh.FactHoldings	SecurityKey (via DimSecurity)
stg.Holdings	MarketValue	dwh.FactHoldings	MarketValue

37. Load Orchestration Dependency

Step	Object	Dependency
1	dwh.DimClient	None — loads from stg.Clients
2	dwh.DimAdvisor	None — loads from stg.Advisors
3	dwh.DimAccount	Requires DimClient (ClientID reference)
4	dwh.DimSecurity	None — loads from stg.Securities
5	dwh.DimBenchmark	None — loads from stg.Benchmarks
6	dwh.FactTrades	Requires DimAccount, DimSecurity
7	dwh.FactHoldings	Requires DimAccount, DimSecurity
8	dwh.FactFees	Requires DimAccount
9	dwh.FactCashflows	Requires DimAccount
10	dwh.FactPerformance	Requires DimAccount, DimBenchmark

38. Phase 4 Summary

Phase 4 defined the SQL foundation of the Wealth Analytics Data Platform, covering schema separation, staging table design, warehouse dimension and fact design, surrogate key strategy, SCD planning, stored procedure loading logic, validation queries, and fact-dimension orchestration order. This phase established the technical SQL foundation required for Azure Data Factory pipeline automation, validation orchestration, and final analytical reporting.

Phase 5 — Azure Data Factory Pipelines, Validation Logic, Logging & Monitoring

39. Azure Data Factory Architecture

39.1 ADF Purpose and Role

Azure Data Factory served as the orchestration layer of the Wealth Analytics Data Platform. ADF was responsible for moving data from ADLS Gen2 Bronze to Azure SQL staging tables, orchestrating SQL stored procedure execution, managing pipeline dependencies, and monitoring ELT execution.

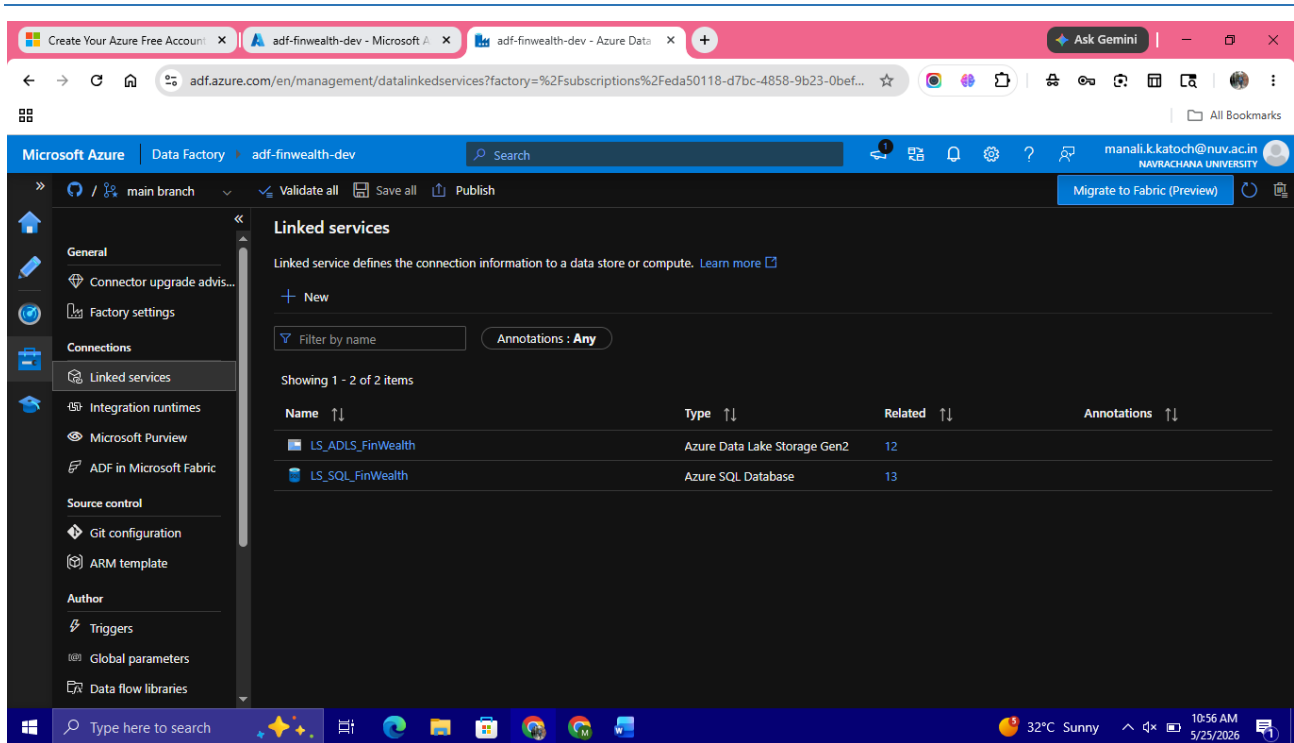
39.2 ADF Configuration

Configuration	Value
Resource Group	rg-finwealth-dev
Data Factory Name	adf-finwealth-dev
Region	Central India
Version	V2
Git Configuration	Configured post-initial development
Networking	Public endpoint
Encryption	Default platform-managed

40. Linked Services

40.1 Linked Service Inventory

Linked Service	Target System	Purpose
LS_ADLS_FinWealth	ADLS Gen2 — stfinwealthanjali	Connect ADF to Bronze CSV files
LS_AzureSQL_FinWealth	Azure SQL — sqldb-finwealth-dev	Connect ADF to staging and DWH tables



40.2 SQL Connectivity Troubleshooting

Issue	Resolution
Firewall blocked connection	Add client IP / allow Azure services in SQL firewall rules
Incorrect server name	Verify fully qualified SQL server endpoint
Invalid credentials	Re-enter SQL authentication username and password
Database not specified	Select the correct database in linked service configuration
Insufficient permissions	Confirm user has table read/write and EXECUTE permissions

41. Dataset Design in ADF

41.1 Dataset Naming Convention

Dataset Type	Naming Pattern
Source CSV Dataset	DS_Bronze_<DatasetName>_CSV
SQL Staging Dataset	DS_SQL_stg_<DatasetName>
SQL DWH Dataset	DS_SQL_dwh_<ObjectName>

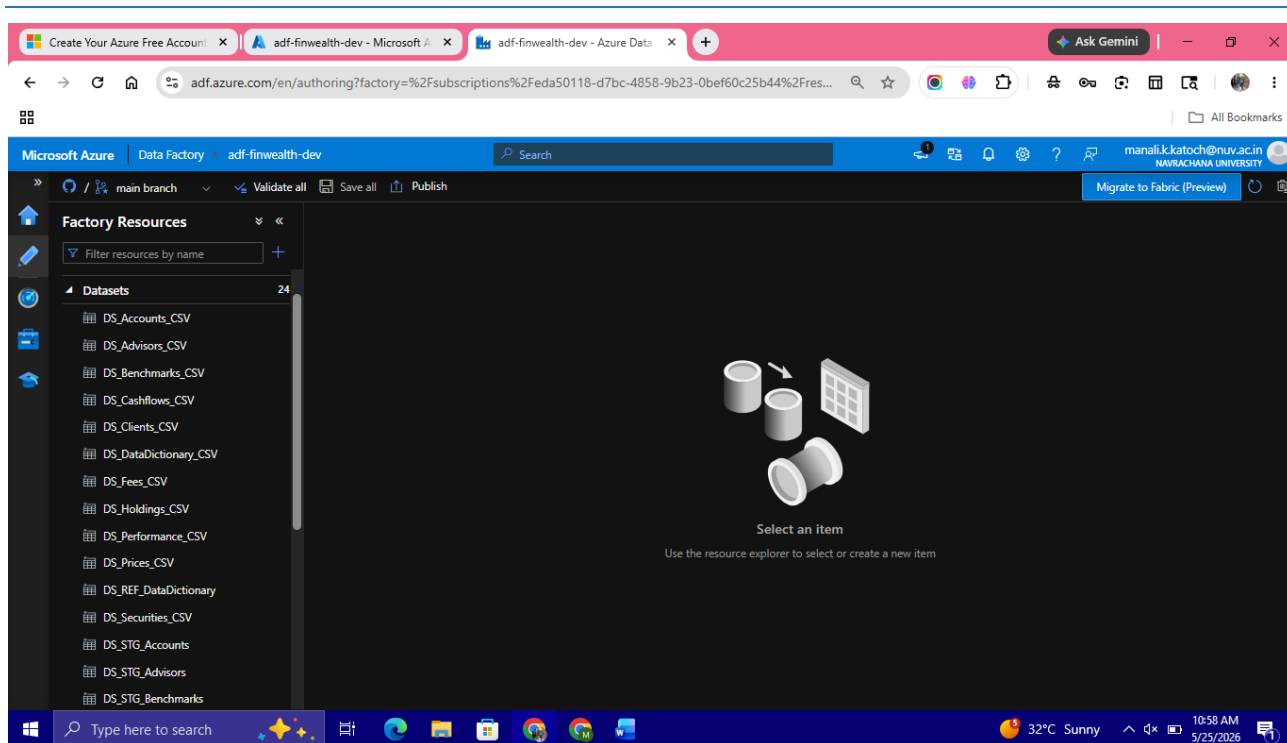


Figure 5.1 — ADF Source and Sink Dataset Configuration

42. Bronze-to-Staging Pipeline Design

42.1 Pipeline Inventory

Pipeline Name	Purpose
PL_Copy_Clients_To_Staging	Load Clients.csv to stg.Clients
PL_Copy_Advisors_To_Staging	Load Advisors.csv to stg.Advisors
PL_Copy_Accounts_To_Staging	Load Accounts.csv to stg.Accounts
PL_Copy_Securities_To_Staging	Load Securities.csv to stg.Securities
PL_Copy_Trades_To_Staging	Load Trades.csv to stg.Trades
PL_Copy_Holdings_To_Staging	Load Holdings.csv to stg.Holdings
PL_Copy_Performance_To_Staging	Load Performance.csv to stg.Performance
PL_Copy_Cashflows_To_Staging	Load Cashflows.csv to stg.Cashflows
PL_Copy_Fees_To_Staging	Load Fees.csv to stg.Fees
PL_Copy_Prices_To_Staging	Load Prices.csv to stg.Prices

42.2 Copy Activity Configuration Standard

Configuration Area	Selected Value
Source	ADLS Gen2 Bronze CSV dataset
Sink	Azure SQL staging table
Activity Type	Copy Activity
Column Mapping	Source columns mapped to staging columns; LoadDate excluded from mapping
Pre-Copy Script	TRUNCATE TABLE stg.<TableName>
Load Mode	Full refresh — staging is truncated on each pipeline run

Note: The pre-copy TRUNCATE statement ensures old staging data is removed before each pipeline run, preventing duplicate loads and simplifying testing. The LoadDate column must not be mapped from the source CSV — the SQL DEFAULT value populates it automatically.

43. Pipeline Validation Steps

Each ADF pipeline followed a structured validation-to-execution sequence:

1. Create pipeline and configure Copy Activity
2. Configure source dataset (Bronze CSV)
3. Configure sink dataset (SQL staging table)
4. Map columns (exclude LoadDate from mapping)
5. Validate pipeline in ADF Studio
6. Publish pipeline
7. Execute using Debug
8. Verify SQL row counts
9. Inspect sample rows with SELECT TOP 10

44. Transformation Pipeline Design

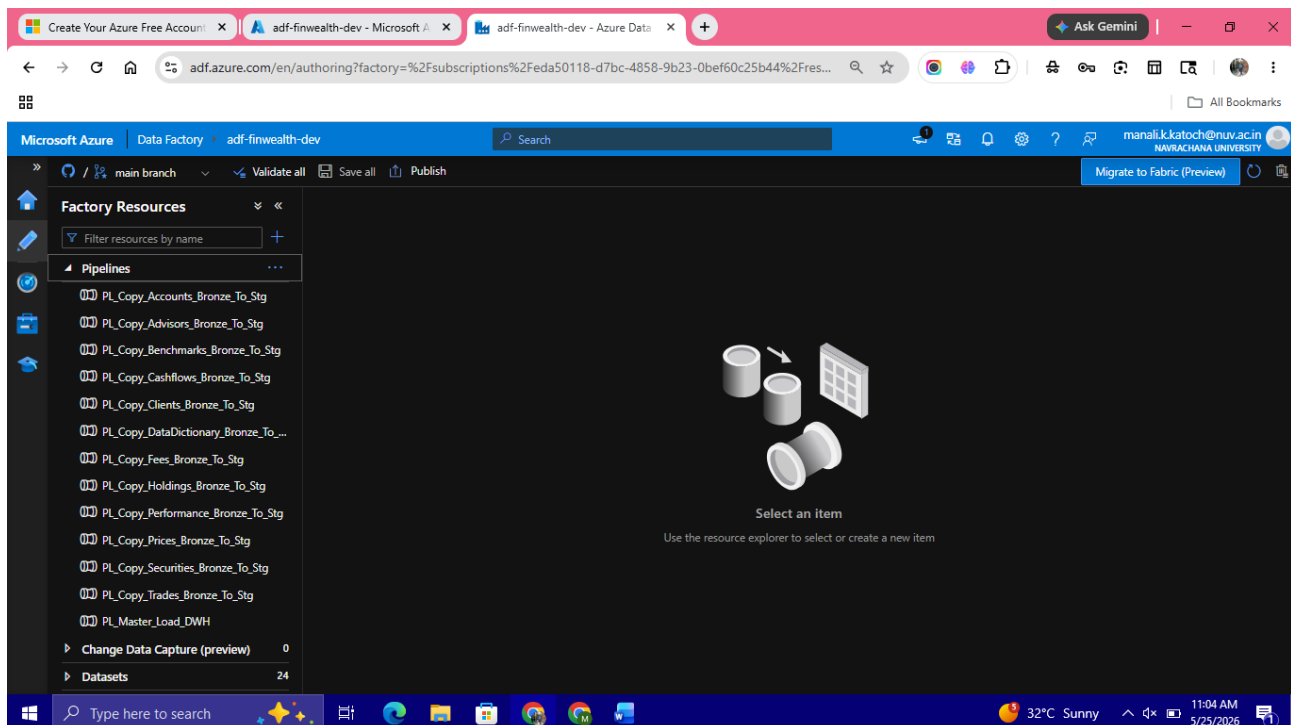
44.1 Dimension Load Pipeline

Pipeline Name	Activity	Stored Procedure
PL_Load_Dimensions	Load Client Dimension	dwh.SP_Load_DimClient

Pipeline Name	Activity	Stored Procedure
PL_Load_Dimensions	Load Advisor Dimension	dwh.SP_Load_DimAdvisor
PL_Load_Dimensions	Load Account Dimension	dwh.SP_Load_DimAccount
PL_Load_Dimensions	Load Security Dimension	dwh.SP_Load_DimSecurity
PL_Load_Dimensions	Load Benchmark Dimension	dwh.SP_Load_DimBenchmark

44.2 Fact Load Pipeline

Pipeline Name	Activity	Stored Procedure
PL_Load_Facts	Load Trades Fact	dwh.SP_Load_FactTrades
PL_Load_Facts	Load Holdings Fact	dwh.SP_Load_FactHoldings
PL_Load_Facts	Load Performance Fact	dwh.SP_Load_FactPerformance
PL_Load_Facts	Load Cashflows Fact	dwh.SP_Load_FactCashflows
PL_Load_Facts	Load Fees Fact	dwh.SP_Load_FactFees



45. Master Pipeline Orchestration

45.1 Master Pipeline: PL_Master_Load_DWH

The master pipeline orchestrated the complete ELT workflow from staging ingestion to warehouse loading, using sequential execution, dependency chaining, and stored procedure orchestration.

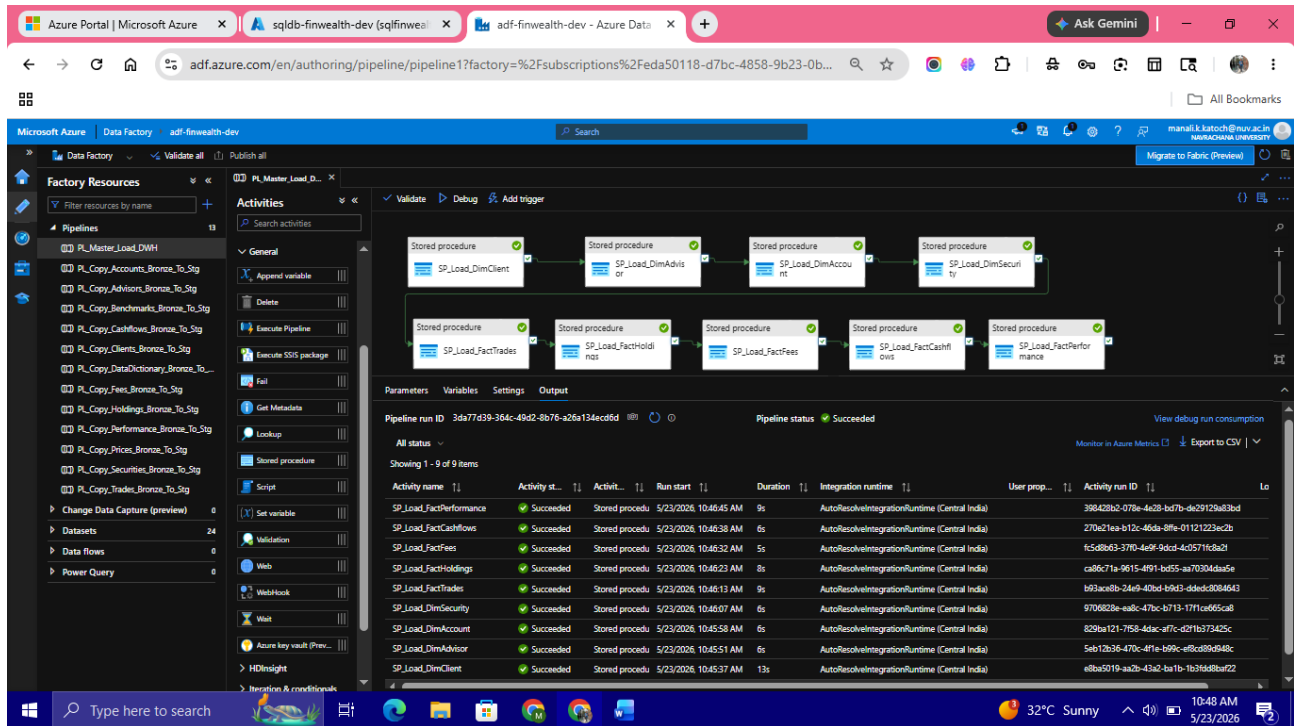


Figure 5.2 — Master ADF Pipeline for End-to-End Warehouse Load

Stage	Activities
Stage 1 — Staging Ingestion	PL_Copy_* pipelines for all 10 datasets (sequential)
Stage 2 — Dimension Loading	PL_Load_Dimensions — executes 5 stored procedures
Stage 3 — Fact Loading	PL_Load_Facts — executes 5 stored procedures after dimensions
Stage 4 — Validation	audit.SP_Validate_StagingData with @PipelineRunID parameter
Stage 5 — Logging	audit.SP_LogPipelineEnd — writes final status to PipelineRunLog

46. Validation Logic

46.1 Validation Framework

Validation Type	Purpose
Row count validation	Confirm source-to-staging load completeness
Null validation	Detect missing mandatory values
Duplicate validation	Detect repeated business keys
Referential integrity	Detect missing parent records in staging
Business rule validation	Detect invalid financial values
Row reconciliation	Compare staging and warehouse load output counts

46.2 Row Reconciliation Queries

```

SELECT
    'Clients' AS EntityName,
    (SELECT COUNT(*) FROM stg.Clients) AS StagingCount,
    (SELECT COUNT(*) FROM dwh.DimClient WHERE IsCurrent = 1) AS
WarehouseCount;

SELECT
    'Trades' AS EntityName,
    (SELECT COUNT(*) FROM stg.Trades) AS StagingCount,
    (SELECT COUNT(*) FROM dwh.FactTrades) AS WarehouseCount;

```

47. Validation Stored Procedure

47.1 audit.ValidationResults Table

```

CREATE TABLE audit.ValidationResults
(
    ValidationID      INT      IDENTITY(1,1) PRIMARY KEY,
    PipelineRunID     VARCHAR(100),
    TableName         VARCHAR(100),
    ValidationType     VARCHAR(100),
    FailedCount        INT,
    ValidationStatus   VARCHAR(20),
    ValidationTime     DATETIME2 DEFAULT SYSDATETIME()
);
GO

```

47.2 audit.SP_Validate_StagingData

Attribute	Detail
Purpose	Centralize staging data quality checks for ADF pipeline orchestration
Source Tables	stg.Clients, stg.Trades (extendable to all staging tables)
Target Table	audit.ValidationResults
Validation Logic	NULL checks and business rule checks; results written with Passed/Failed status
Business Rules	Null ClientID = data quality failure; negative/zero trade amounts = business rule violation

```

CREATE OR ALTER PROCEDURE audit.SP_Validate_StagingData
    @PipelineRunID VARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    -- Null ClientID Check
    INSERT INTO audit.ValidationResults
    (PipelineRunID, TableName, ValidationType, FailedCount, ValidationStatus)
    SELECT
        @PipelineRunID,
        'stg.Clients',
        'Null ClientID Check',
        COUNT(*),
        CASE WHEN COUNT(*) = 0 THEN 'Passed' ELSE 'Failed' END
    FROM stg.Clients
    WHERE ClientID IS NULL;

    -- Invalid Trade Amount Check
    INSERT INTO audit.ValidationResults
    (PipelineRunID, TableName, ValidationType, FailedCount, ValidationStatus)
    SELECT
        @PipelineRunID,
        'stg.Trades',
        'Invalid Trade Amount Check',
        COUNT(*),
        CASE WHEN COUNT(*) = 0 THEN 'Passed' ELSE 'Failed' END
    FROM stg.Trades
    WHERE Quantity <= 0
       OR Price <= 0
       OR TradeAmount <= 0;
END;
GO

```

Note: ADF executes this procedure using a Stored Procedure Activity and passes the pipeline run ID dynamically via the expression @pipeline().RunId.

48. Logging Framework

48.1 audit.PipelineRunLog Table

```
CREATE TABLE audit.PipelineRunLog
(
    LogID                INT IDENTITY(1,1) PRIMARY KEY,
    PipelineRunID        VARCHAR(100),
    PipelineName         VARCHAR(200),
    ActivityName         VARCHAR(200),
    SourceName           VARCHAR(200),
    TargetName           VARCHAR(200),
    RowsRead             INT NULL,
    RowsWritten          INT NULL,
    Status               VARCHAR(50),
    ErrorMessage         VARCHAR(MAX) NULL,
    StartTime            DATETIME2,
    EndTime              DATETIME2,
    LoggedAt             DATETIME2 DEFAULT SYSDATETIME()
);
GO
```

48.2 audit.SP_LogPipelineStart

```
CREATE OR ALTER PROCEDURE audit.SP_LogPipelineStart
(
    @PipelineRunID VARCHAR(100),
    @PipelineName  VARCHAR(200),
    @ActivityName  VARCHAR(200),
    @SourceName    VARCHAR(200),
    @TargetName    VARCHAR(200),
    @StartTime     DATETIME2
)
AS
BEGIN
    INSERT INTO audit.PipelineRunLog
    (PipelineRunID, PipelineName, ActivityName, SourceName, TargetName,
    Status, StartTime)
    VALUES
    (@PipelineRunID, @PipelineName, @ActivityName, @SourceName, @TargetName,
    'Started', @StartTime);
END;
GO
```

48.3 audit.SP_LogPipelineEnd

```

CREATE OR ALTER PROCEDURE audit.SP_LogPipelineEnd
(
    @PipelineRunID VARCHAR(100),
    @ActivityName   VARCHAR(200),
    @RowsRead       INT,
    @RowsWritten    INT,
    @Status          VARCHAR(50),
    @ErrorMessage   VARCHAR(MAX) = NULL,
    @EndTime         DATETIME2
)
AS
BEGIN
    UPDATE audit.PipelineRunLog
    SET
        RowsRead       = @RowsRead,
        RowsWritten    = @RowsWritten,
        Status          = @Status,
        ErrorMessage   = @ErrorMessage,
        EndTime         = @EndTime
    WHERE PipelineRunID = @PipelineRunID
        AND ActivityName = @ActivityName;
END;
GO

```

48.4 ADF Dynamic Content References

Expression	Value Captured
@pipeline().RunId	Unique pipeline run identifier
@pipeline().Pipeline	Pipeline name
@activity('Copy_Clients_To_Staging').output.rowsRead	Rows read by Copy Activity
@activity('Copy_Clients_To_Staging').output.rowsCopied	Rows written to staging
@activity('Copy_Clients_To_Staging').error.message	Error message on failure path

49. Monitoring Strategy

Monitoring Area	Purpose
Pipeline status	Identify success or failure at run level
Activity duration	Detect slow-running or stalled activities
Rows read / written	Validate ingestion completeness
Error messages	Diagnose root cause of failures

Monitoring Area	Purpose
Retry attempts	Identify unstable pipeline execution
Stored procedure failures	Detect SQL-side transformation issues

49.1 Operational Monitoring Queries

```

SELECT TOP 50 * FROM audit.PipelineRunLog ORDER BY LoggedAt DESC;
SELECT TOP 50 * FROM audit.ValidationResults ORDER BY ValidationTime DESC;

SELECT ValidationStatus, COUNT(*) AS CheckCount
FROM audit.ValidationResults
GROUP BY ValidationStatus;

```

50. Error Handling & Retry Strategy

Failure Scenario	Possible Cause	Resolution
Source file not found	Incorrect Bronze container path	Verify container and file name in dataset
SQL connection failure	Firewall or credential issue	Update firewall rules / re-enter credentials
Column mapping failure	Source-target column mismatch	Fix ADF column mapping configuration
Datatype conversion error	Invalid CSV values	Stage as VARCHAR; cleanse in SQL transformation
Stored procedure failure	SQL logic or syntax error	Test stored procedure independently in SSMS
FK mismatch on fact load	Dimensions not loaded first	Validate dimension data, then re-run fact load

51. Phase 5 Summary

Phase 5 defined the Azure Data Factory implementation architecture for the Wealth Analytics Data Platform, covering ADF setup, linked services, source and sink datasets, Bronze-to-staging copy pipelines, staging validation, stored procedure execution, dimension and fact load orchestration, master pipeline design, validation logic, logging tables, monitoring strategy, error handling, retry policy, and trigger strategy. ADF was established as the orchestration layer while SQL remained the transformation and validation engine.

Phase 6 — Business Rules, Cost Optimization & Enterprise Best Practices

52. Business Rules Framework

52.1 Business Rule Categories

Rule Category	Purpose
Structural Rules	Ensure schema integrity across all staging tables
Referential Rules	Maintain valid entity relationships
Financial Rules	Validate monetary values and financial logic
Analytical Rules	Support accurate and consistent reporting
Data Quality Rules	Detect incomplete or invalid data
Warehouse Rules	Ensure dimensional loading consistency
Operational Rules	Govern pipeline execution sequencing

53. Entity-Level Business Rules

53.1 Client Business Rules

Rule	Validation Query	Business Impact
Every client must have a unique ClientID	GROUP BY ClientID HAVING COUNT(*) > 1	Duplicate clients distort AUM and profitability calculations
Every active client must map to a valid advisor	LEFT JOIN stg.Advisors WHERE AdvisorID IS NULL	Missing advisor mapping breaks performance and commission reporting
Risk profiles must belong to controlled values (Conservative / Moderate / Aggressive)	WHERE RiskProfile NOT IN ('Conservative','Moderate','Aggressive')	Non-standard values corrupt risk-based analytics and segmentation

53.2 Account Business Rules

Rule	Validation Query	Reason
Every account must belong to a valid client	LEFT JOIN stg.Clients WHERE ClientID IS NULL	Accounts without clients violate ownership structure
Account status must be standardized (Active / Inactive / Closed)	WHERE Status NOT IN ('Active','Inactive','Closed')	Non-standard status values break operational reporting

53.3 Trade Business Rules

Rule	Validation Logic	Business Impact
Trade quantity must be greater than zero	WHERE Quantity <= 0	Invalid quantities distort trading volume and portfolio holdings
Trade price must be positive	WHERE Price <= 0	Zero or negative prices indicate source corruption or mapping error
Trade amount must be positive	WHERE TradeAmount <= 0	Negative amounts invalidate AUM and cashflow calculations
Every trade must reference a valid security	LEFT JOIN stg.Securities WHERE SecurityID IS NULL	Unresolved security references produce orphaned fact records

53.4 Holdings, Performance & Financial Business Rules

Entity	Rule	Validation
Holdings	Market value cannot be negative	WHERE MarketValue < 0
Holdings	Grain must remain one row per account-security-date	GROUP BY AccountID, SecurityID, HoldingDate HAVING COUNT(*) > 1
Performance	Benchmark references must resolve to valid benchmarks	LEFT JOIN stg.Benchmarks WHERE BenchmarkCode IS NULL
Performance	Return values must be within realistic bounds (-100% to 1000%)	WHERE ReturnPct < -100 OR ReturnPct > 1000
Fees	Fee amounts cannot be negative	WHERE FeeAmount < 0

Entity	Rule	Validation
Cashflows	AccountID, Amount, and FlowDate must all be present	WHERE AccountID IS NULL OR Amount IS NULL OR FlowDate IS NULL

54. Data Quality Governance Rules

Governance Rule	Dataset	Mandatory Columns
Null Governance — mandatory fields must not be NULL	Clients	ClientID
Null Governance	Accounts	AccountID, ClientID
Null Governance	Trades	TradeID, AccountID, SecurityID
Null Governance	Holdings	AccountID, SecurityID
Null Governance	Performance	AccountID
Duplicate Governance — business keys must be unique in master entities	All master datasets	Per business key column
Referential Integrity — all FK relationships validated before warehouse load	All transactional datasets	AccountID, SecurityID, BenchmarkCode

55. Cost Optimization Strategy

Service / Component	Decision	Reason
Azure Standard Storage + LRS	Selected	Lowest-cost redundancy — sufficient for dev environment
ADF Copy Activity	Preferred	Lightweight, cost-safe file movement
SQL Stored Procedures	Preferred	No additional compute cost — transformations run within SQL tier
Azure Databricks	Avoided	Unnecessary cluster compute overhead
Azure Synapse Analytics	Avoided	Excessive complexity and cost for project scope

Service / Component	Decision	Reason
Mapping Data Flow Debug	Avoided	Silently consumes Integration Runtime cluster resources
Multiple Integration Runtimes	Avoided	Unnecessary operational overhead

56. Enterprise Best Practices Summary

56.1 Architecture Best Practices

Best Practice	Purpose
Implement Medallion Architecture	Layered governance and data zone separation
Preserve Bronze immutability	Auditability, lineage, and reprocessing support
Separate storage and compute	Independent scalability and cost control
Use SQL for transformations	Maintainable, reusable, version-controllable logic
Use star schema design	Simplified reporting and Power BI optimization

56.2 Security & Governance

Setting	Configuration
Anonymous Access	Disabled on all storage containers
Public Blob Access	Disabled
Encryption	Platform-managed encryption enabled by default
Bronze Layer Governance	Bronze data must never be directly modified

56.3 Troubleshooting Reference

Issue	Likely Cause
SQL connection failure	Firewall rule not configured for Azure services
Column mapping failure	Source and target column name or type mismatch
Null insertion error	Mandatory staging field contains NULL value from source
FK failure in fact load	Dimension table not loaded before fact procedure

Issue	Likely Cause
Stored procedure failure	SQL syntax error or business logic issue — test SP independently

57. Phase 6 Summary

Phase 6 established enterprise business rules, warehouse governance standards, financial validation logic, cost optimization strategy, security considerations, and troubleshooting methodology. This phase elevated the implementation from a basic ETL pipeline to a governed enterprise-style Data Engineering solution with clearly defined quality standards, financial correctness rules, and cost-aware architecture decisions.

Appendix D - SQL Queries Used During Practical Implementation

This appendix contains the final SQL queries that were executed successfully during practical implementation. These scripts were used for business view creation, NULL validation, schema inspection during debugging, and data quality validation.

D.1 Final NULL Validation Across Business Views

```
SELECT 'vw_ClientPortfolioSummary' AS ViewName, COUNT(*) AS NullRows
FROM dwh.vw_ClientPortfolioSummary
WHERE ClientID IS NULL OR ClientName IS NULL OR Segment IS NULL OR Country IS
NULL OR TotalAUM IS NULL
UNION ALL
SELECT 'vw_AdvisorPerformance', COUNT(*)
FROM dwh.vw_AdvisorPerformance
WHERE AdvisorID IS NULL OR AdvisorName IS NULL OR Region IS NULL OR TotalAUM IS
NULL
UNION ALL
SELECT 'vw_AUM_By_Region', COUNT(*)
FROM dwh.vw_AUM_By_Region
WHERE Region IS NULL OR TotalAUM IS NULL
UNION ALL
SELECT 'vw_FeeRevenueAnalysis', COUNT(*)
FROM dwh.vw_FeeRevenueAnalysis
WHERE ClientID IS NULL OR AdvisorID IS NULL OR Region IS NULL OR TotalFeeRevenue
IS NULL
UNION ALL
SELECT 'vw_TradeActivity', COUNT(*)
FROM dwh.vw_TradeActivity
WHERE ClientID IS NULL OR SecurityID IS NULL OR AssetClass IS NULL OR Sector IS
NULL OR TradeType IS NULL
UNION ALL
SELECT 'vw_ProductMixAnalysis', COUNT(*)
FROM dwh.vw_ProductMixAnalysis
WHERE AssetClass IS NULL OR Sector IS NULL OR TotalAUM IS NULL;
```

D.2 vw_ProductMixAnalysis

```
CREATE OR ALTER VIEW dwh.vw_ProductMixAnalysis
AS
SELECT
    ISNULL(ds.AssetClass, 'Unknown') AS AssetClass,
    ISNULL(ds.Sector, 'Unknown') AS Sector,
    ds.Currency,
    ds.Region,
    COUNT(DISTINCT ds.SecurityKey) AS TotalSecurities,
    SUM(fh.MarketValue) AS TotalAUM,
    SUM(ft.TradeAmount) AS TotalTradeAmount,
    SUM(ft.Quantity) AS TotalQuantity
FROM dwh.DimSecurity ds
LEFT JOIN dwh.Fact_Holdings fh
    ON ds.SecurityKey = fh.SecurityKey
LEFT JOIN dwh.Fact_Trades ft
    ON ds.SecurityKey = ft.SecurityKey
GROUP BY
    ISNULL(ds.AssetClass, 'Unknown'),
```

```
        ISNULL(ds.Sector, 'Unknown'),
        ds.Currency,
        ds.Region;
GO
```

```
SELECT TOP 20 *
FROM dwh.vw_ProductMixAnalysis
ORDER BY TotalAUM DESC;
```

D.3 vw_TradeActivity

```
CREATE OR ALTER VIEW dwh.vw_TradeActivity
AS
SELECT
    dc.ClientID,
    dc.ClientName,
    dc.Segment,
    dc.Country,
    ds.SecurityID,
    ds.SecurityName,
    ds.AssetClass,
    ISNULL(ds.Sector, 'Unknown') AS Sector,
    ft.TradeDate,
    ISNULL(ft.TradeType, 'Unknown') AS TradeType,
    COUNT(ft.TradeKey) AS TotalTrades,
    SUM(ft.Quantity) AS TotalQuantity,
    SUM(ft.TradeAmount) AS TotalTradeAmount,
    AVG(ft.Price) AS AvgTradePrice
FROM dwh.Fact_Trades ft
LEFT JOIN dwh.DimAccount acc
    ON ft.AccountKey = acc.AccountKey
LEFT JOIN dwh.DimClient dc
    ON acc.ClientID = dc.ClientID
LEFT JOIN dwh.DimSecurity ds
    ON ft.SecurityKey = ds.SecurityKey
GROUP BY
    dc.ClientID,
    dc.ClientName,
    dc.Segment,
    dc.Country,
    ds.SecurityID,
    ds.SecurityName,
    ds.AssetClass,
    ds.Sector,
    ft.TradeDate,
    ft.TradeType;
GO

SELECT TOP 20 *
FROM dwh.vw_TradeActivity
ORDER BY TotalTradeAmount DESC;
```

D.4 vw_FeeRevenueAnalysis

```
CREATE OR ALTER VIEW dwh.vw_FeeRevenueAnalysis
AS
```



```

SELECT
    dc.ClientID,
    dc.ClientName,
    ISNULL(dc.Segment, 'Unknown') AS Segment,
    ISNULL(dc.Country, 'Unknown') AS Country,
    adv.AdvisorID,
    adv.AdvisorName,
    ISNULL(adv.Region, 'Unknown') AS Region,
    COUNT(DISTINCT acc.AccountID) AS TotalAccounts,
    ISNULL(SUM(fee.FeeAmount), 0) AS TotalFeeRevenue,
    ISNULL(AVG(fee.FeeAmount), 0) AS AvgFeePerTransaction,
    ISNULL(MAX(fee.FeeAmount), 0) AS HighestFee,
    ISNULL(MIN(fee.FeeAmount), 0) AS LowestFee
FROM dwh.Fact_Fees fee
LEFT JOIN dwh.DimAccount acc
    ON fee.AccountKey = acc.AccountKey
LEFT JOIN dwh.DimClient dc
    ON acc.ClientID = dc.ClientID
LEFT JOIN dwh.DimAdvisor adv
    ON dc.PrimaryAdvisorID = adv.AdvisorID
GROUP BY
    dc.ClientID,
    dc.ClientName,
    ISNULL(dc.Segment, 'Unknown'),
    ISNULL(dc.Country, 'Unknown'),
    adv.AdvisorID,
    adv.AdvisorName,
    ISNULL(adv.Region, 'Unknown');
GO

SELECT TOP 20 *
FROM dwh.vw_FeeRevenueAnalysis
ORDER BY TotalFeeRevenue DESC;

```

D.5 vw_AUM_By_Region

```

CREATE OR ALTER VIEW dwh.vw_AUM_By_Region
AS
SELECT
    ISNULL(adv.Region, 'Unknown') AS Region,
    COUNT(DISTINCT adv.AdvisorID) AS TotalAdvisors,
    COUNT(DISTINCT dc.ClientID) AS TotalClients,
    COUNT(DISTINCT acc.AccountKey) AS TotalAccounts,
    SUM(hold.MarketValue) AS TotalAUM,
    AVG(perf.ReturnPct) AS AvgPortfolioReturn
FROM dwh.DimAdvisor adv
LEFT JOIN dwh.DimClient dc
    ON adv.AdvisorID = dc.PrimaryAdvisorID
LEFT JOIN dwh.DimAccount acc
    ON dc.ClientID = acc.ClientID
LEFT JOIN dwh.Fact_Holdings hold
    ON acc.AccountKey = hold.AccountKey
LEFT JOIN dwh.Fact_Performance perf
    ON acc.AccountKey = perf.AccountKey
GROUP BY
    ISNULL(adv.Region, 'Unknown')

```

```
GO
```

```
SELECT *
FROM dwh.vw_AUM_By_Region
ORDER BY TotalAUM DESC;
```

D.6 vw_AdvisorPerformance

```
CREATE OR ALTER VIEW dwh.vw_AdvisorPerformance
AS
SELECT
    adv.AdvisorID,
    adv.AdvisorName,
    ISNULL(adv.Region, 'Unknown') AS Region,
    adv.ExperienceYears,

    COUNT(DISTINCT dc.ClientID) AS TotalClients,
    COUNT(DISTINCT acc.AccountKey) AS TotalAccounts,

    ISNULL(SUM(hold.MarketValue), 0) AS TotalAUM,
    ISNULL(SUM(fee.FeeAmount), 0) AS TotalFeesGenerated,
    ISNULL(AVG(perf.ReturnPct), 0) AS AvgPortfolioReturn

FROM dwh.DimAdvisor adv

LEFT JOIN dwh.DimClient dc
    ON adv.AdvisorID = dc.PrimaryAdvisorID

LEFT JOIN dwh.DimAccount acc
    ON dc.ClientID = acc.ClientID

LEFT JOIN dwh.Fact_Holdings hold
    ON acc.AccountKey = hold.AccountKey

LEFT JOIN dwh.Fact_Fees fee
    ON acc.AccountKey = fee.AccountKey

LEFT JOIN dwh.Fact_Performance perf
    ON acc.AccountKey = perf.AccountKey

GROUP BY
    adv.AdvisorID,
    adv.AdvisorName,
    ISNULL(adv.Region, 'Unknown'),
    adv.ExperienceYears;

GO

SELECT TOP 20 *
FROM dwh.vw_AdvisorPerformance
```

```
ORDER BY TotalAUM DESC;
```

D.7 Schema Inspection Queries Used During Debugging

```
SELECT COLUMN_NAME
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dwh'
AND TABLE_NAME = 'DimAdvisor';
```

```
SELECT COLUMN_NAME
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dwh'
AND TABLE_NAME = 'DimAccount';
```

D.8 vw_ClientPortfolioSummary

```
CREATE OR ALTER VIEW dwh.vw_ClientPortfolioSummary
AS
SELECT
    dc.ClientID,
    dc.ClientName,
    dc.Segment,
    dc.RiskProfile,
    ISNULL(dc.Country, 'Unknown') AS Country,

    COUNT(DISTINCT da.AccountKey) AS TotalAccounts,
    COUNT(DISTINCT fh.SecurityKey) AS TotalSecurities,

    ISNULL(SUM(fh.MarketValue), 0) AS TotalAUM,
    ISNULL(SUM(ff.FeeAmount), 0) AS TotalFees

FROM dwh.DimClient dc

LEFT JOIN dwh.DimAccount da
    ON dc.ClientID = da.ClientID

LEFT JOIN dwh.Fact_Holdings fh
    ON da.AccountKey = fh.AccountKey

LEFT JOIN dwh.Fact_Fees ff
    ON da.AccountKey = ff.AccountKey

GROUP BY
    dc.ClientID,
    dc.ClientName,
    dc.Segment,
    dc.RiskProfile,
    ISNULL(dc.Country, 'Unknown');
GO

SELECT TOP 20 *
```

```
FROM dwh.vw_ClientPortfolioSummary
ORDER BY TotalAUM DESC;
```

D.9 Data Quality Audit Table and Stored Procedure

```
CREATE TABLE audit.DataQualityResults
(
    CheckID          INT IDENTITY(1,1) PRIMARY KEY,
    CheckName        VARCHAR(200),
    TableName        VARCHAR(100),
    CheckType        VARCHAR(100),
    IssueCount       INT,
    Status           VARCHAR(20),
    CheckDate        DATETIME2 DEFAULT SYSDATETIME()
);
GO

CREATE OR ALTER PROCEDURE audit.usp_Run_DataQualityChecks
AS
BEGIN

SET NOCOUNT ON;

TRUNCATE TABLE audit.DataQualityResults;

-----
-- NULL CHECK : DimClient
-----

INSERT INTO audit.DataQualityResults
(
    CheckName,
    TableName,
    CheckType,
    IssueCount,
    Status
)

SELECT
'Null ClientID Check',
'dwh.DimClient',
'NULL CHECK',
COUNT(*),

CASE
    WHEN COUNT(*) = 0 THEN 'PASSED'
    ELSE 'FAILED'
END

FROM dwh.DimClient
WHERE ClientID IS NULL;
```

```
-----  
-- NULL CHECK : DimAdvisor  
-----  
  
INSERT INTO audit.DataQualityResults  
(  
    CheckName,  
    TableName,  
    CheckType,  
    IssueCount,  
    Status  
)  
  
SELECT  
'Null AdvisorID Check',  
'dwh.DimAdvisor',  
'NULL CHECK',  
COUNT(*),  
  
CASE  
    WHEN COUNT(*) = 0 THEN 'PASSED'  
    ELSE 'FAILED'  
END  
  
FROM dwh.DimAdvisor  
WHERE AdvisorID IS NULL;
```

```
-----  
-- DUPLICATE CHECK : DimAccount  
-----  
  
INSERT INTO audit.DataQualityResults  
(  
    CheckName,  
    TableName,  
    CheckType,  
    IssueCount,  
    Status  
)  
  
SELECT  
'Duplicate AccountID Check',  
'dwh.DimAccount',  
'DUPLICATE CHECK',  
COUNT(*),  
  
CASE  
    WHEN COUNT(*) = 0 THEN 'PASSED'
```

```
        ELSE 'FAILED'
    END

FROM
(
    SELECT AccountID
    FROM dwh.DimAccount
    GROUP BY AccountID
    HAVING COUNT(*) > 1
) A;

-----

-- FACT VALIDATION : FactTrades
-----

INSERT INTO audit.DataQualityResults
(
    CheckName,
    TableName,
    CheckType,
    IssueCount,
    Status
)

SELECT
'Null Trade Amount Check',
'dwh.Fact_Trades',
'NULL CHECK',
COUNT(*),

CASE
    WHEN COUNT(*) = 0 THEN 'PASSED'
    ELSE 'FAILED'
END

FROM dwh.Fact_Trades
WHERE TradeAmount IS NULL;

-----

-- FACT VALIDATION : FactHoldings
-----

INSERT INTO audit.DataQualityResults
(
    CheckName,
    TableName,
    CheckType,
    IssueCount,
    Status
)

```

```
SELECT
    'Negative Market Value Check',
    'dwh.Fact_Holdings',
    'BUSINESS RULE CHECK',
    COUNT(*) ,

CASE
    WHEN COUNT(*) = 0 THEN 'PASSED'
    ELSE 'FAILED'
END

FROM dwh.Fact_Holdings
WHERE MarketValue < 0;

END;
GO

EXEC audit.usp_Run_DataQualityChecks;

SELECT *
FROM audit.DataQualityResults;
```

Appendix — Project Repository & References

A.1 GitHub Repository

The complete source code, SQL scripts, ADF pipeline configurations, and project documentation for the Wealth Analytics Data Platform are available in the GitHub repository below. The repository serves as the primary reference for all implementation artifacts delivered as part of this enterprise Azure Data Engineering project.

GitHub Repository: <https://github.com/Anjali2718/azure-finwealth-data-platform>

The repository contains the following project deliverables:

- SQL scripts — Staging, Dimension, Fact, Audit schema creation and stored procedures
- ADF pipeline configurations — Copy, transformation, and master orchestration pipelines
- ER Diagram — Enterprise Star Schema (draw.io format)
- Project documentation — This enterprise implementation report